

Azure Databricks For Data Engineers

Project on Formula1 Racing



Ramesh Retnasamy
Data Engineer/ Machine Learning Engineer



<https://www.linkedin.com/in/ramesh-retnasamy/>

About Me

About this course



Azure Databricks



PySpark



Spark SQL



Delta Lake



Azure Databricks



Azure Data Lake Storage Gen2



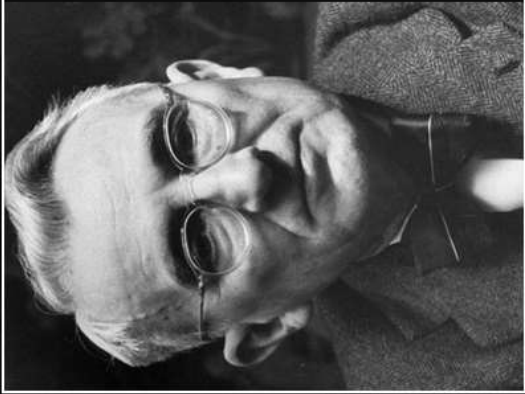
Azure Data Factory



Azure Key Vault



PowerBI



Learning is an active process. We learn by doing.. Only knowledge that is used sticks in your mind.

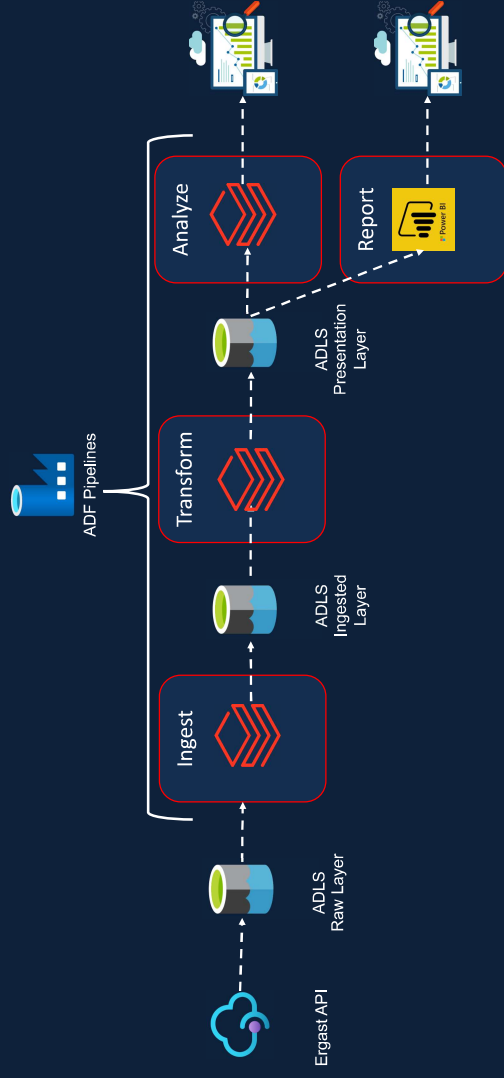
— Dale Carnegie —

AZ QUOTES

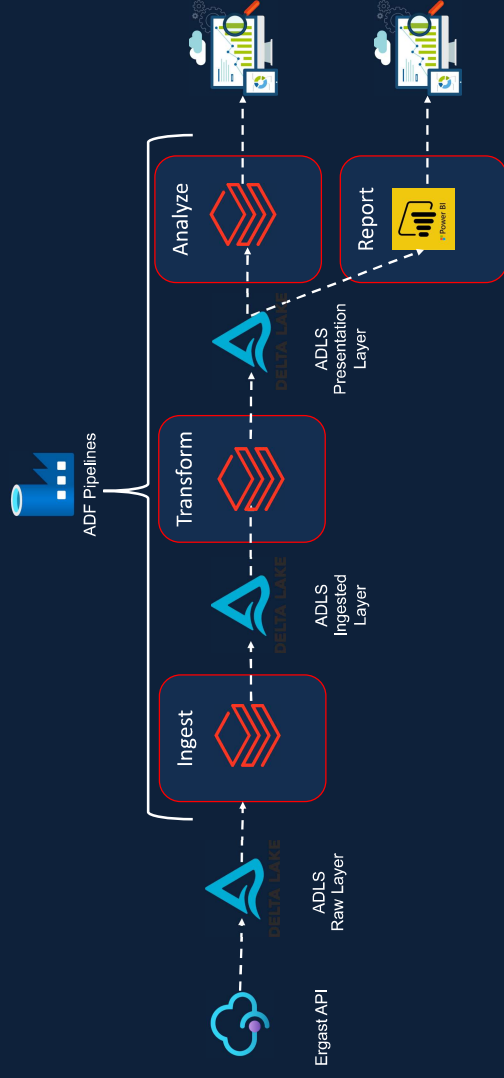


Formula1 Cloud Data Platform

Formula1 Cloud Data Platform



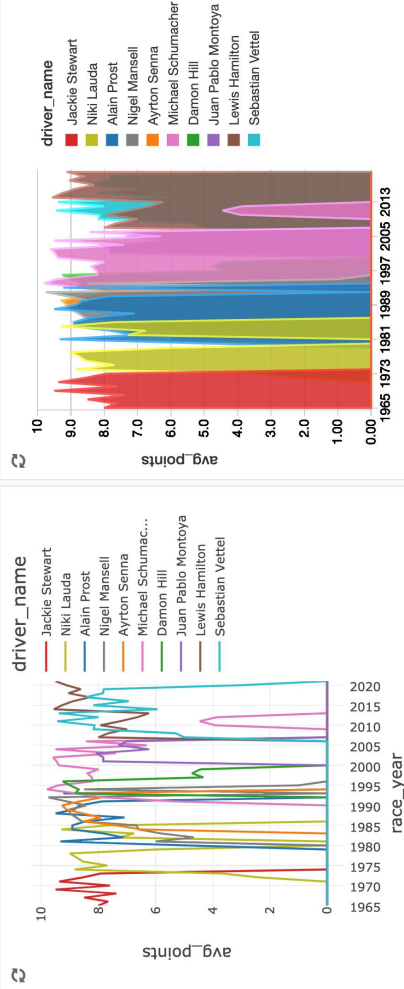
Formula1 Cloud Data Platform



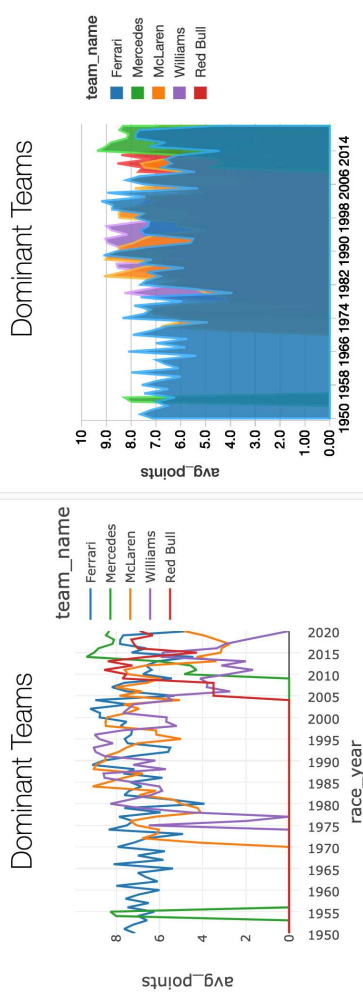
Formula1 Cloud Data Platform

Formula1 Cloud Data Platform

Dominant Formula 1 Drivers of All Time



Dominant Formula 1 Teams of All Time



Who is this course for

University students

IT Developers from other disciplines

AWS/ GCP/ On-prem Data Engineers

Data Architects

Who is this course not for

You are not interested in hands-on learning approach

Your only focus is Azure Data Engineering Certification

You want to Learn Spark ML or Streaming

You want to Learn Scala or Java

Pre-requisites

All code and step-by-step instructions provided

Basic Python programming knowledge required

Basic SQL knowledge required

Cloud fundamentals will be beneficial, but not mandatory

Azure Account

Our Commitments

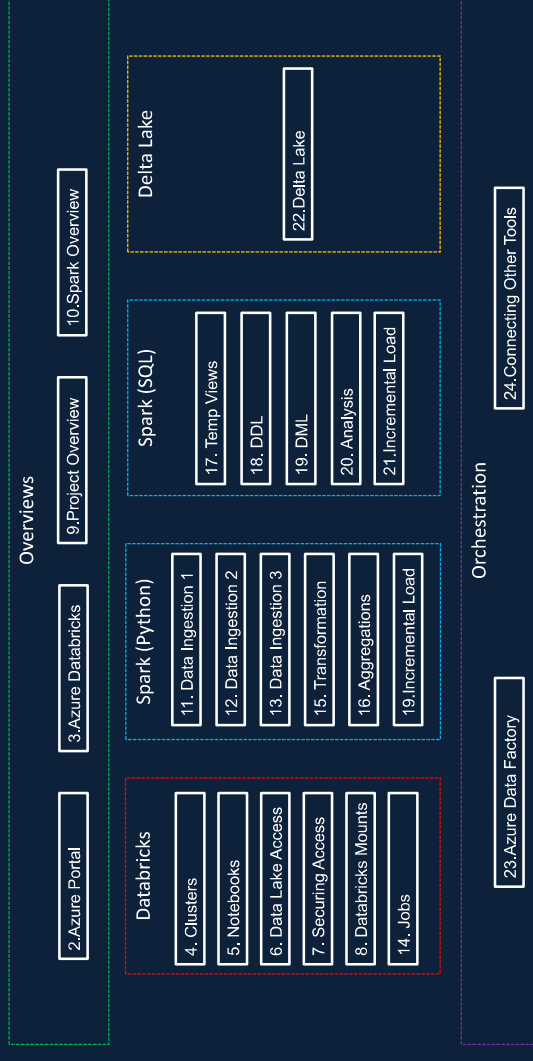
Ask Questions, I will answer 😊

Keeping the course up to date

Udemy life time access

Udemy 30 day money back guarantee

Course Structure



Introduction to Azure Databricks



Azure Databricks

Apache Spark



Apache Spark is a lightning-fast unified analytics engine for big data processing and machine learning

100% Open source under Apache License

Simple and easy to use APIs

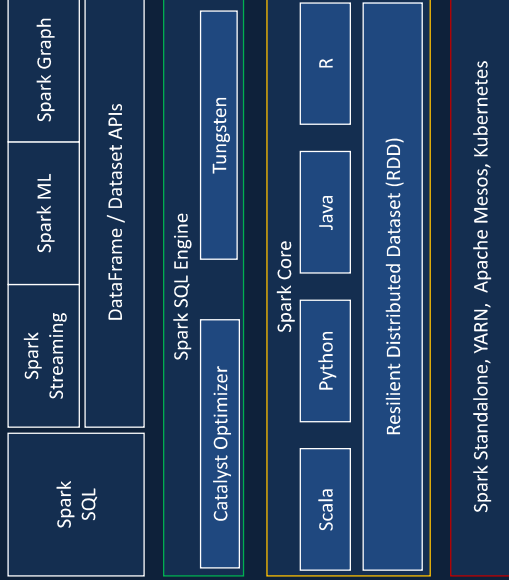
In-memory processing engine

Distributed computing Platform

Unified engine which supports SQL, streaming, ML and graph processing



Apache Spark Architecture



Databricks



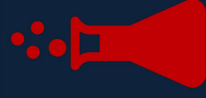
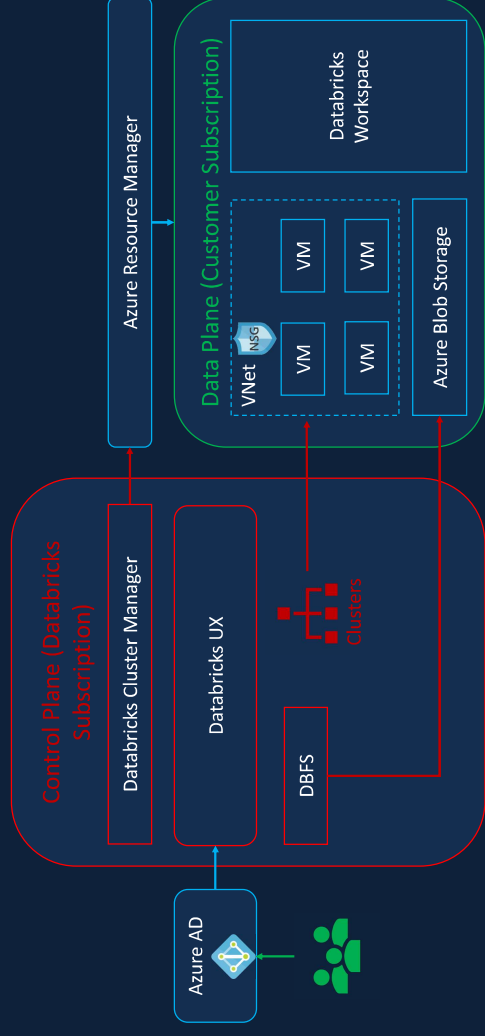
Azure Databricks



Azure Databricks

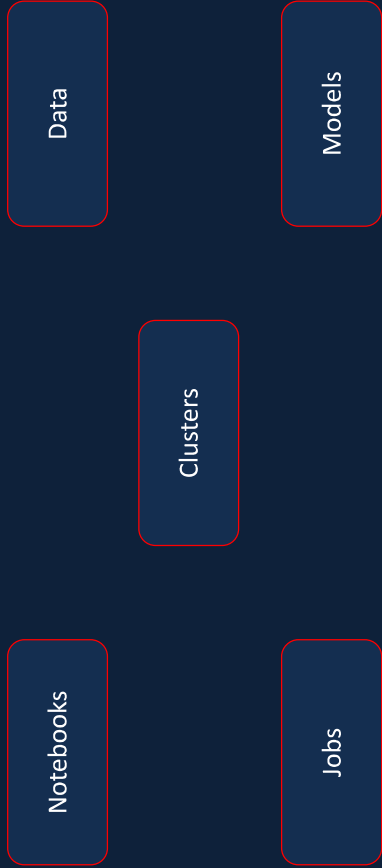


Azure Databricks Architecture



Creating Azure Databricks Service

Databricks Workspace Components

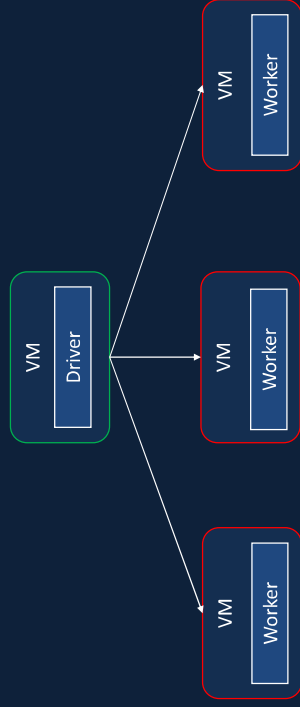


Databricks Clusters

- What is Databricks Cluster
- Cluster Types
- Cluster Configuration
- Creating a cluster
- Pricing
- Cost Control
- Cluster Pools
- Cluster Policy



Databricks Cluster



Cluster Types

All Purpose	Job Cluster
Created manually	Created by Jobs
Persistent	Terminated at the end of the job
Suitable for interactive workloads	Suitable for automated workloads
Shared among many users	Isolated just for the job
Expensive to run	Cheaper to run

Cluster Configuration

Cluster Configuration ⓘ

Policy ⓘ

Unrestricted

Multi node

Single node

Access mode ⓘ

Single user access ⓘ

Single user

Ramesh Retnasamy (ez.adm1...

Performance

Databricks runtime version ⓘ

Runtime: 11.3 LTS (Scala 2.12, Spark 3.3.0)

Use Photon Acceleration ⓘ

Worker type ⓘ

Standard_DS3_v2

14 GB Memory, 4 Cores

Driver type

Same as worker

Enable autoscaling ⓘ

Terminate after

120 minutes of inactivity ⓘ

Min workers

2

Max workers

8

Spot instances ⓘ

Cluster Configuration

Single/ Multi Node

Multi Node

Single Node

Cluster Configuration

Single/ Multi Node

Access Mode

Single User

Only One User Access
Supports Python, SQL, Scala, R

Shared

Multiple User Access
Only available in Premium. Supports Python, SQL

No Isolation Shared

Multiple User Access
Supports Python, SQL

Custom

Legacy Configuration

Cluster Configuration

Single/ Multi Node

Access Mode

Databricks Runtime

Databricks Runtime

Scala, Java, Python, R
Spark
Databricks Runtime Libraries
GPU Libraries
Other Databricks Services
Delta Lake

Databricks Runtime ML

Everything from Databricks runtime
Popular ML Libraries (PyTorch, Keras, TensorFlow, XGBoost etc)

Photon Runtime

Everything from Databricks runtime
Photon Engine

Databricks Runtime Light

Runtime option for only jobs not requiring advanced features

Cluster Configuration

Single/ Multi Node

Access Mode

Databricks Runtime

Auto Termination

- Auto Termination
- Terminates the cluster after X minutes of inactivity
- Default value for Single Node and Standard clusters is 120 minutes
- Users can specify a value between 10 and 10000 mins as the duration

Cluster Configuration

Single/ Multi Node

Access Mode

Databricks Runtime

Auto Termination

Auto Scaling

- Auto Scaling
- User specifies the min and max work nodes
- Auto scales between min and max based on the workload
- Not recommended for streaming workloads

Cluster Configuration

Single/ Multi Node

Access Mode

Databricks Runtime

Auto Termination

Auto Scaling

Cluster VM Type/ Size

- Memory Optimized
- Compute Optimized
- Storage Optimized
- General Purpose
- GPU Accelerated

Cluster Configuration

Single/ Multi Node

Access Mode

Databricks Runtime

Auto Termination

Auto Scaling

Cluster VM Type/ Size

Cluster Policy

Cluster Configuration

Policy

Unrestricted

Multi node

Single node

Access mode

Single user access

Single user

Ramesh Retnasamy (az.adm...

Performance

Databricks runtime version

Runtime: 11.3 LTS (Scala 2.12, Spark 3.3.0)

☐ Use Photon Acceleration

Worker type

Standard_DS3_V2

Min workers

2

Max workers

8

Driver type

Same as worker

14 GB Memory, 4 Cores

14 GB Memory, 4 Cores

☒ Enable autoscaling

120 minutes of inactivity

☒ Terminate after

120 minutes of inactivity

Cluster Configuration

Single/ Multi Node

Access Mode

Databricks Runtime

Auto Termination

Auto Scaling

Cluster VM Type/ Size

Cluster Policy

Cluster Configuration

Policy

Personal Compute

Single user access

Ramesh Retnasamy (az.admi@outlook.com)

Performance

Databricks runtime version

Runtime: 11.3 LTS ML (Scala 2.12, Spark 3.3.0)

Node type

Standard_DS3_v2

14 GB Memory, 4 Cores

☒ Terminate after 20 minutes of inactivity

Cluster Configuration

Single/ Multi Node

Access Mode

Databricks Runtime

Auto Termination

Auto Scaling

Cluster VM Type/ Size

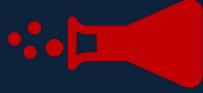
Cluster Policy

Simplifies the user interface

Enables standard users to create clusters

Achieves cost control

Only available on premium tier



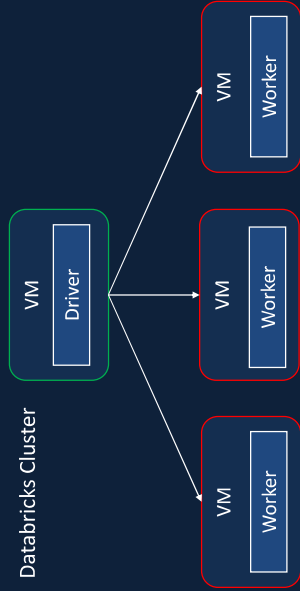
Creating Databricks Cluster

Azure Databricks Cluster Pricing

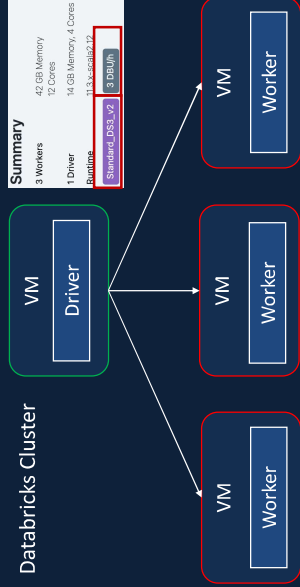


Azure Databricks Pricing Factors

Azure Databricks Pricing Calculation



- Workload (All Purpose/ Jobs/ SQL/ Photon)
- Tier (Premium/ Standard)
- VM Type (General Purpose/ GPU/ Optimized)
- Purchase Plan (Pay As You Go/ Pre-Purchase)



Summary	
3 Workers	42 GB Memory
1 Driver	12 Cores
Runtime	14 GB Memory, 4 Cores
	11.3x-scala2.12
	Standard_DS3_v2
	3 DBU/h

A Databricks Unit (DBU) is a normalized unit of processing power on the Databricks Lakehouse Platform used for measurement and pricing purposes

Azure Databricks Pricing Calculation

Azure Databricks Pricing Calculation

No of DBU	X	Price based on workload/ tier	=	DBU Cost	+	Driver Node Cost	+	Worker Node Cost	=	Total Cost of the Cluster
1 (Driver)	X	Price of VM	=							
No of workers	X	Price of VM	=							

Single Node Cluster

Summary	
1 Driver	14 GB Memory, 4 Cores
Runtime	11.3x-scala2.12
	Standard_DS3_v2
	0.75 DBU/h

Workload - All Purpose

Pricing Tier - Premium

VM Type - General Purpose Standard DS3_V2

Purchase Plan - Pay As You Go

Azure Databricks Pricing Calculation

Workload	DBU prices—standard tier	DBU prices—premium tier
All-Purpose Compute**	\$0.40/DBU-hour	\$0.55/DBU-hour
Jobs Compute**	\$0.15/DBU-hour	\$0.30/DBU-hour
Jobs Light Compute	\$0.07/DBU-hour	\$0.22/DBU-hour
SQL Compute	-	\$0.22/DBU-hour
SQL Pro Compute	-	\$0.37/DBU-hour
Serverless SQL	-	\$0.475/DBU-hour

<https://azure.microsoft.com/en-gb/pricing/details/databricks/>

Azure Databricks Pricing Calculation

Instance	vCPU(s)	RAM	Temporary storage	Pay as you go	1 year savings plan	3 year savings plan	Spot	Add to estimate
DS1 v2	1	3.5 GiB	7 GiB	\$0.0878 ^{hour}	\$0.0760 ^{hour} ~13% savings	\$0.0600 ^{hour} ~31% savings	\$0.0105 ^{hour} ~87% savings	+
DS2 v2	2	7 GiB	14 GiB	\$0.1760 ^{hour}	\$0.1524 ^{hour} ~13% savings	\$0.1203 ^{hour} ~31% savings	\$0.0211 ^{hour} ~87% savings	+
DS3 v2	4	14 GiB	28 GiB	\$0.3510 ^{hour}	\$0.3040 ^{hour} ~13% savings	\$0.2399 ^{hour} ~31% savings	\$0.0422 ^{hour} ~87% savings	+
DS4 v2	8	28 GiB	56 GiB	\$0.7020 ^{hour}	\$0.6080 ^{hour} ~13% savings	\$0.4798 ^{hour} ~31% savings	\$0.0843 ^{hour} ~87% savings	+
DS5 v2	16	56 GiB	112 GiB	\$1.4050 ^{hour}	\$1.2169 ^{hour} ~13% savings	\$0.9603 ^{hour} ~31% savings	\$0.1687 ^{hour} ~87% savings	+

<https://azure.microsoft.com/en-gb/pricing/details/virtual-machines/linux/#pricing>

Azure Databricks Pricing Calculation

Summary			
1 Driver	14 GB Memory, 4 Cores		
Runtime	11.3 x scala2.12		
Standard_DS3_v2	0.75 DBU/h		

No of DBU0.75

x

Price based on workload/ tier0.55

=

DBU Cost0.4125

1 (Driver)1

x

Price of VM0.3510

=

Driver Node Cost0.3510

No of workers0

x

Price of VM

=

Worker Node Cost0.00

=

Total Cost of the Cluster\$0.7635 per hour

Estimated cost for doing the course

Azure Data Lake Storage

Azure Data Factory

Azure Databricks Job Cluster

Azure Databricks Cluster Pool

Azure Databricks All Purpose Cluster

- \$0.76 per hour for a small single node cluster on premium tier
- Depends on the number of hours in use
- Past student experience – 20 to 30 hours to complete the course
- Past *Pay As You Go* student experience – \$15 to \$25
- Within the credit offered by Free / Student Subscription

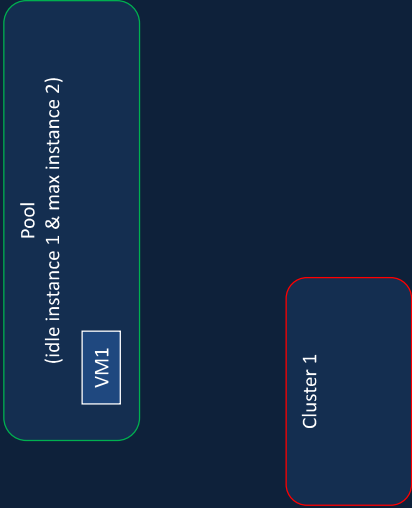
Cost Control

Service	Action to be taken
Azure Data Lake Storage	None – Cost Negligible
Azure Data Factory	None - Billed only for execution of the pipeline
Azure Databricks Job Cluster	None - Destroyed once the job completes
Azure Databricks Cluster Pool	Delete the cluster pool at end of the lesson
Azure Databricks All Purpose Cluster	Set Auto Termination to 20 minutes

Set budget alerts on your subscription

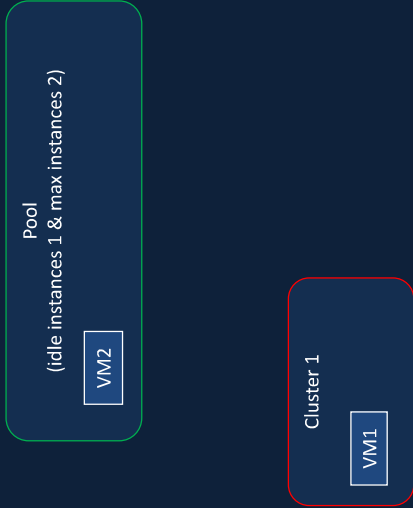
Cluster Pools

- Single/ Multi Node Access Mode
- Databricks Runtime
- Auto Termination
- Auto Scaling
- Cluster VM Type/ Size
- Cluster Policy
- Cluster Pool



Cluster Pools

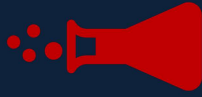
- Single/ Multi Node Access Mode
- Databricks Runtime
- Auto Termination
- Auto Scaling
- Cluster VM Type/ Size
- Cluster Policy
- Cluster Pool



Cluster Pools

- Single/ Multi Node Access Mode
- Databricks Runtime
- Auto Termination
- Auto Scaling
- Cluster VM Type/ Size
- Cluster Policy
- Cluster Pool





Creating Cluster Pool



Cluster Policy

Cluster Policy



- Hide Attributes
- Fix Values
- Set Default Values

- Simple User Interface
- Achieve Cost Control
- Standardize Cluster Configs
- Empowers standard users

Cluster Policy

Public Preview (December 2022)

Only available on Premium Tier

Databricks Notebooks

Create Cluster Policy



What's a notebook

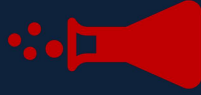
Creating a notebook

Magic Commands

Databricks Utilities

Import Project Solution Notebooks

Creating Notebooks



Magic Commands



Databricks Utilities



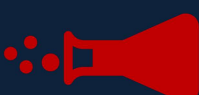
File System Utilities

Secrets Utilities

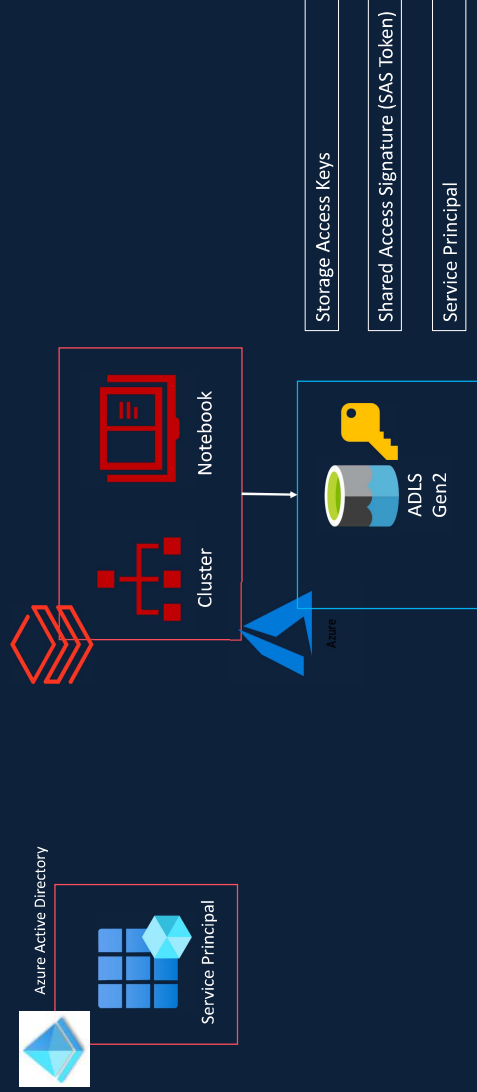
Widget Utilities

Notebook Workflow Utilities

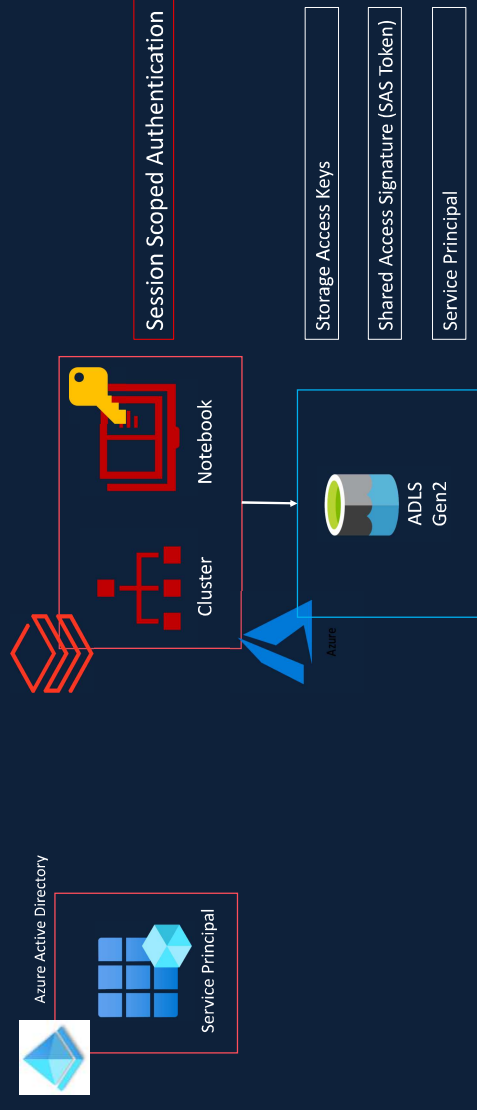
Databricks Utilities



Access Azure Data Lake - Section Overview

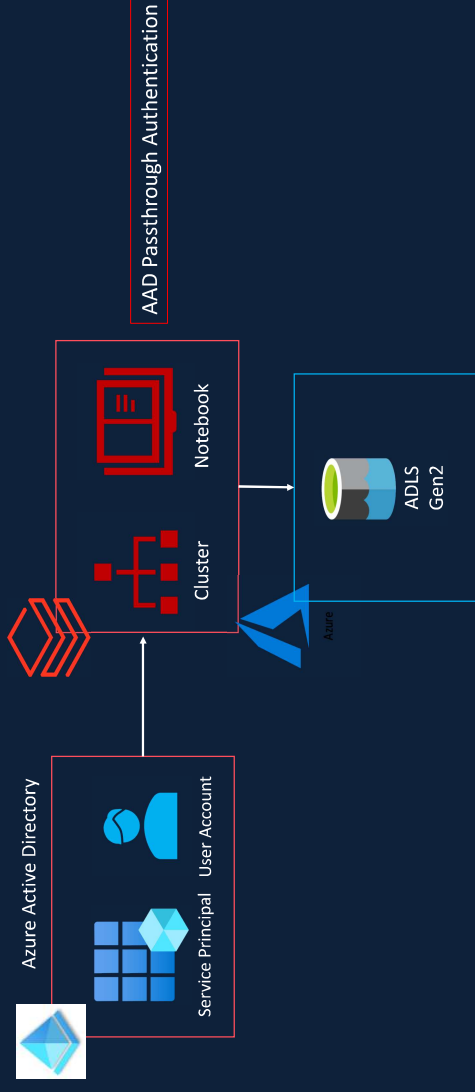
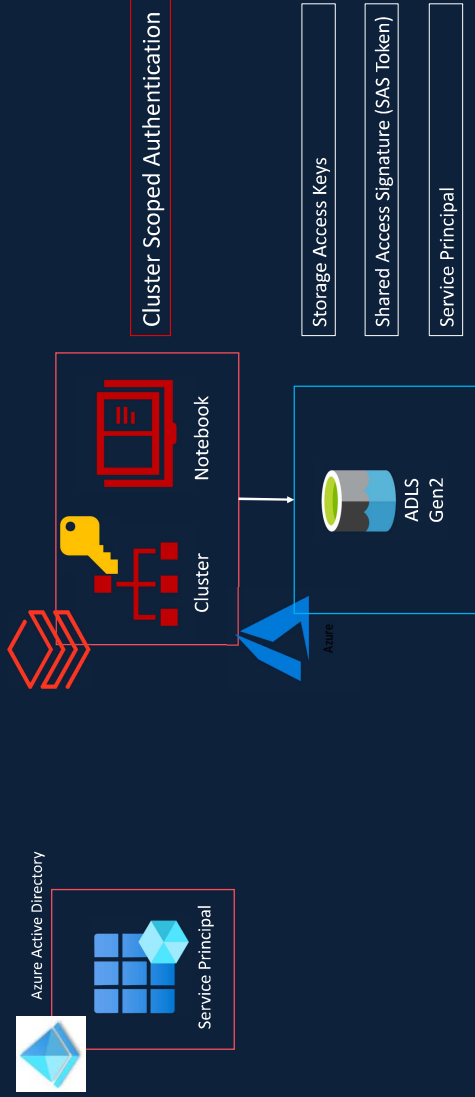


Access Azure Data Lake - Section Overview



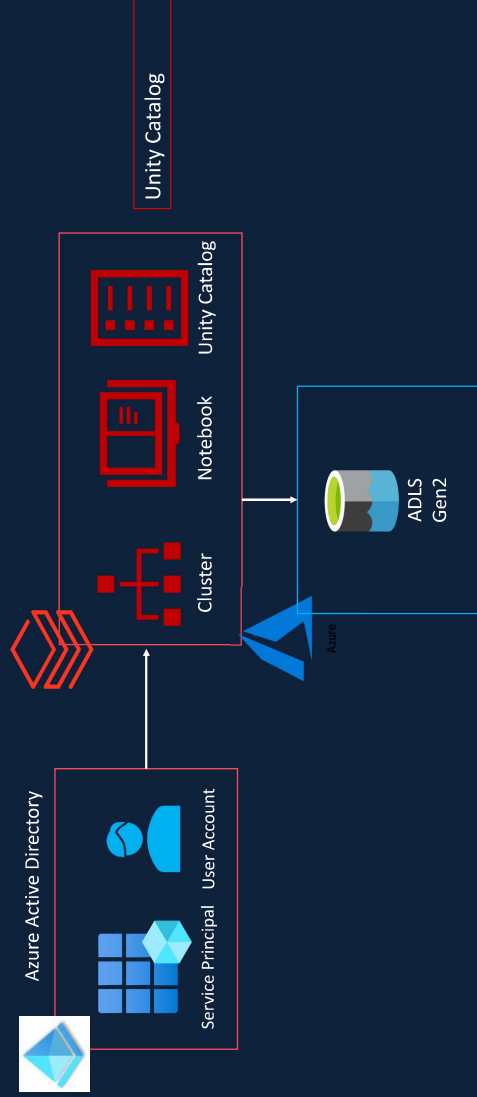
Access Azure Data Lake - Section Overview

Access Azure Data Lake - Section Overview



Access Azure Data Lake - Section Overview

Access Azure Data Lake - Section Overview



- Create Azure Data Lake Gen2 Storage
- Access Data Lake using Access Keys
- Access Data Lake using SAS Token
- Access Data Lake using Service Principal
- Using Cluster Scoped Authentication
- Access Data Lake using AAD Credential Pass-through
- Recommended approach for the course

Create Azure Data Lake Gen2 Storage (ADLS Gen2)



Access Azure Data Lake Gen2 using Access Keys



Authenticate Using Access Keys



ADLS Gen2

Each storage account comes with 2 keys

Gives full Access to the storage account

Keys can be rotated (regenerated)

Access Keys – Spark Configuration



Azure
Databricks



Access Key



Azure Data
Lake Gen2

```
spark.conf.set("fs.azure.account.key<storage-account>.dfs.core.windows.net", "<access key>")
```

```
spark.conf.set(
```

```
"fs.azure.account.key" formula1dl.dfs.core.windows.net",
```

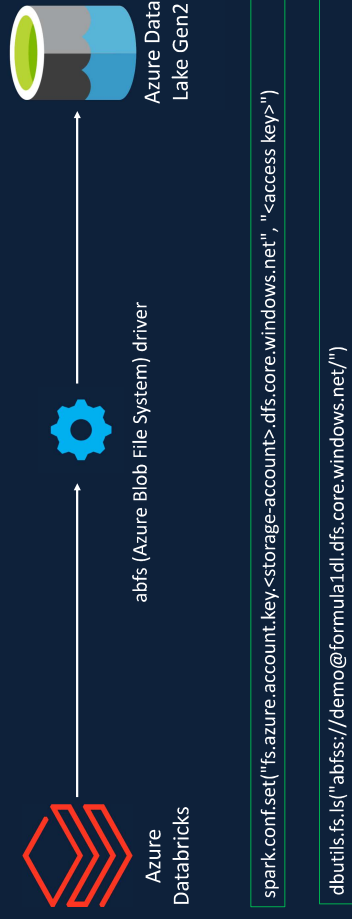
```
"B0R0yW+LaxV39N0JZ7XWRSW50imUGp2IKdE65NrBHRJ9UHLfQLYJN+J+Qunhev+YL8+CwPLenWn+ASTg8bFJg==" )
```

Access Keys – abfs driver

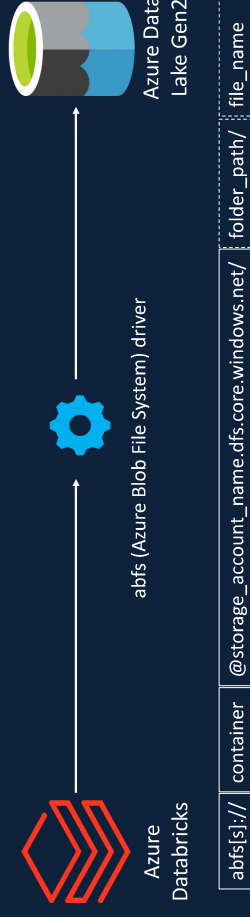


[Microsoft Documentation - https://learn.microsoft.com/en-us/azure/storage/blobs/data-lake-storage-abfs-driver](https://learn.microsoft.com/en-us/azure/storage/blobs/data-lake-storage-abfs-driver)

Authenticate Using Access Keys



Access Keys – abfs driver



abfs://demo@formula1dl.dfs.core.windows.net/

abfs://demo@formula1dl.dfs.core.windows.net/test/

abfs://demo@formula1dl.dfs.core.windows.net/test/circuits.csv

Access Azure Data Lake

using

Shared Access Signature (SAS Token)



Shared Access Signature



- Provides fine grained access to the storage
- Restrict access to specific resource types/ services
- Allow specific permissions
- Restrict access to specific time period
- Limit access to specific IP addresses
- Recommended access pattern for external clients

Microsoft Documentation - <https://learn.microsoft.com/en-us/azure/storage/common/storage-sas-overview>

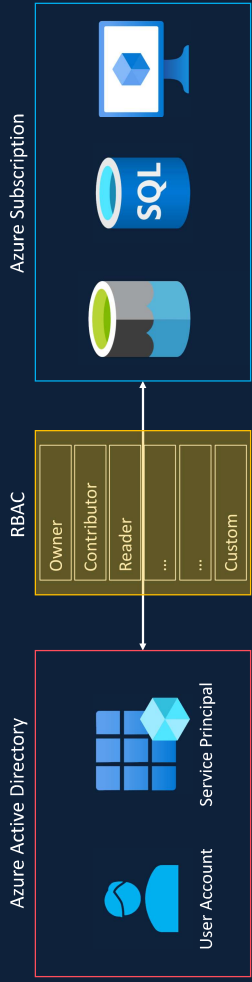
Shared Access Signature



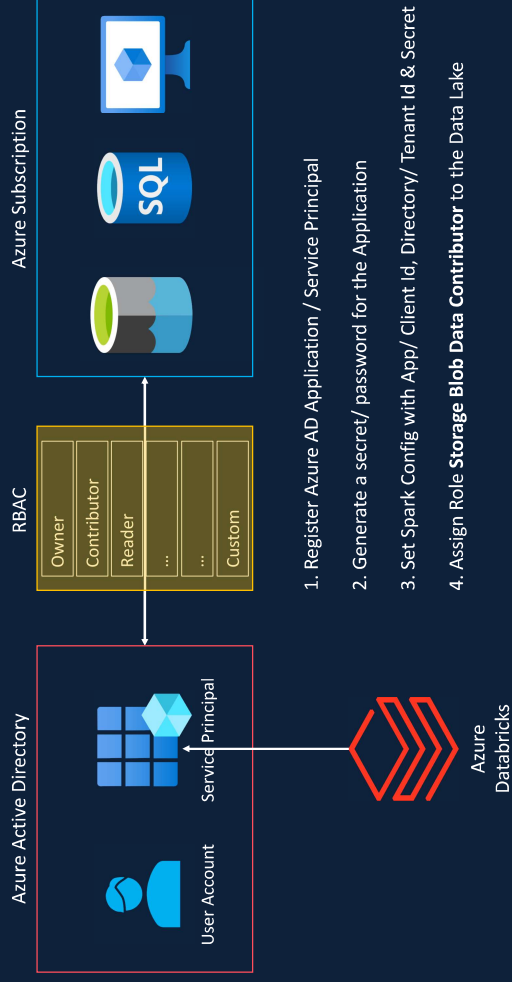
```
spark.conf.set("is.azure.account.auth.type:<storage-account>.dfs.core.windows.net",  
"SAS")  
spark.conf.set("is.azure.sas.token.provider.type:<storage-account>.dfs.core.windows.net",  
"org.apache.hadoop.fs.azurebfs.sas.FixedSASTokenProvider")  
spark.conf.set("is.azure.sas.fixed.token:<storage-account>.dfs.core.windows.net", "<token>")
```

Service Principal

Access Azure Data Lake
using
Service Principal



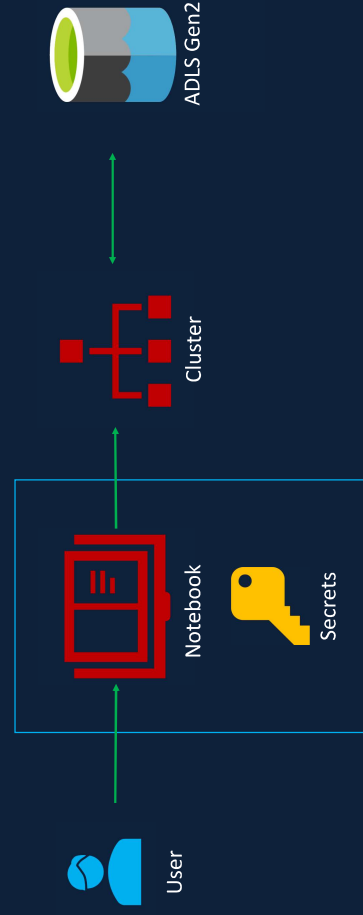
Service Principal



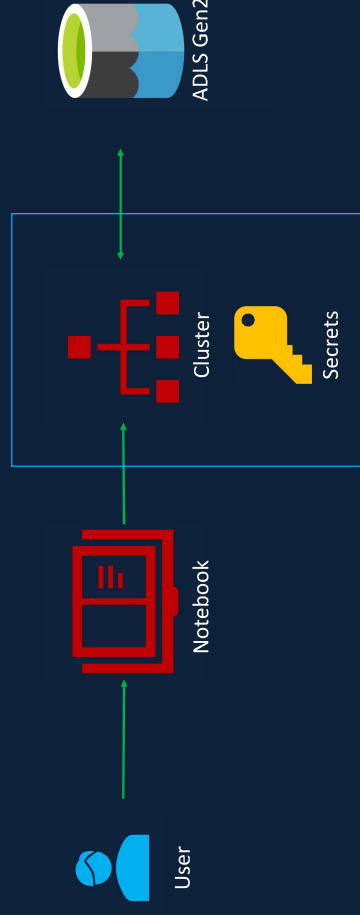
Cluster Scoped Authentication



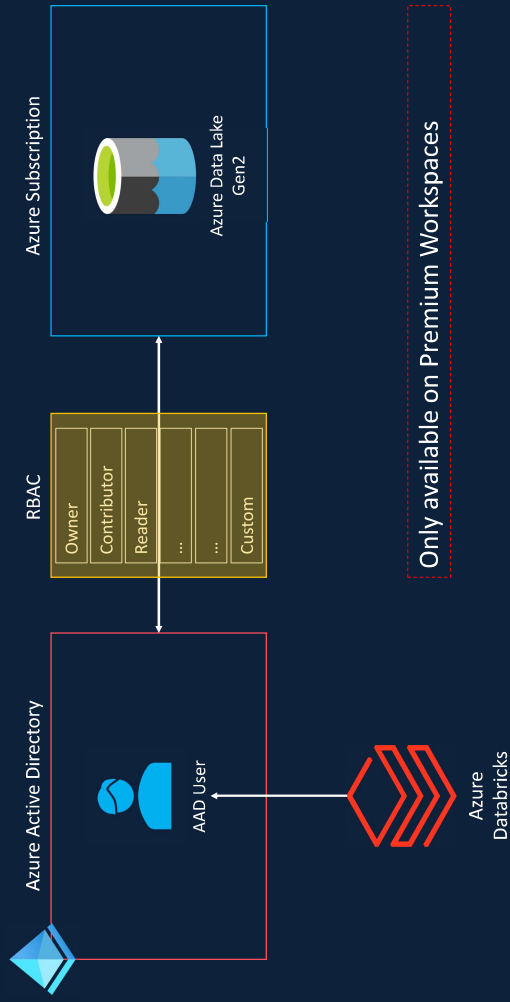
Session Scoped Authentication



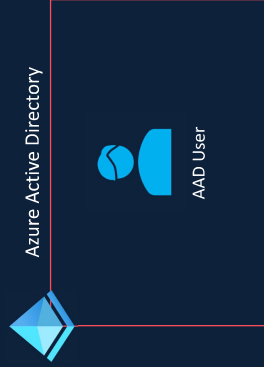
Cluster Scoped Authentication



AAD Credential Passthrough

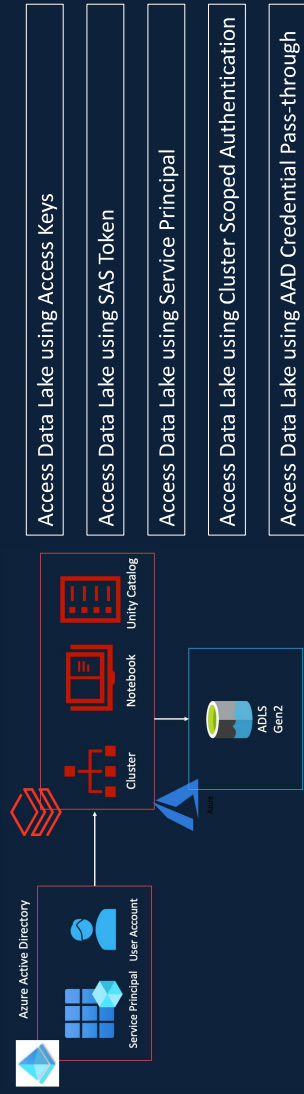


AAD Credential Passthrough



Recommended Access Pattern

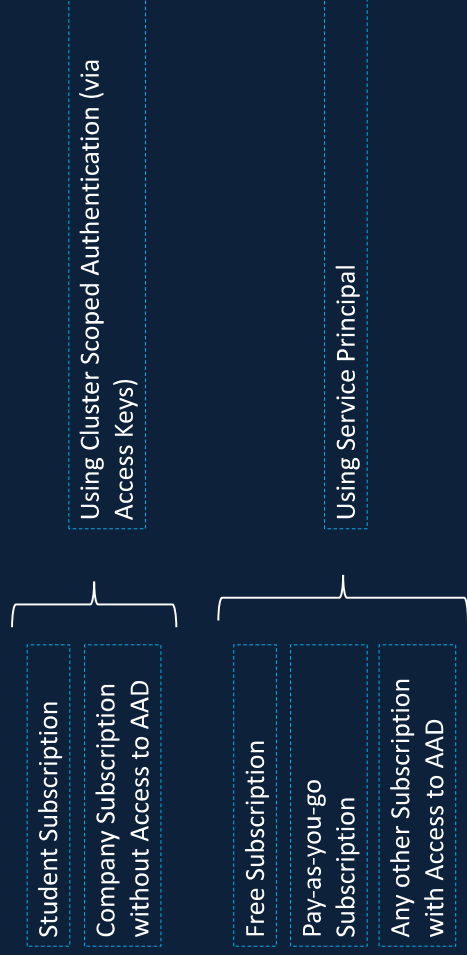
Recommended Access Pattern For The Course



Securing Credentials & Secrets

Databricks Mounts

Recommended Access Pattern



Securing Secrets



Databricks Secret Scope

Azure Key Vault

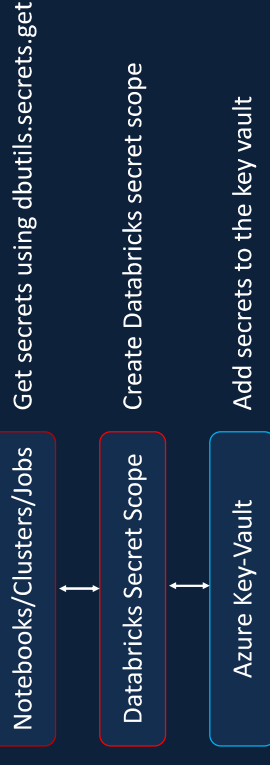
Securing Secrets – Section Overview

Secret scopes help store the credentials securely and reference them in notebooks, clusters and jobs when required

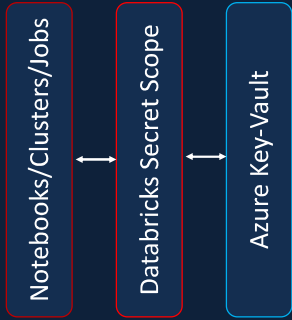
Databricks backed Secret Scope

Azure Key-vault backed Secret Scope

Securing Secrets – Section Overview



Securing Secrets – Section Overview



- Create Azure Key Vault
- Create Databricks Secret Scope
- Databricks Secrets Utility Overview
- Implement Secrets to Notebooks
- Implement Secrets to Clusters

Creating Azure Key Vault



Creating Secret Scope

Databricks Secrets Utility (dbutils.secrets)

Implement Secrets Utility in
Databricks Notebooks

Implement Secrets Utility in
Databricks Notebooks
(Assignment)

Implement Secrets Utility in
Databricks Clusters

Databricks Mounts

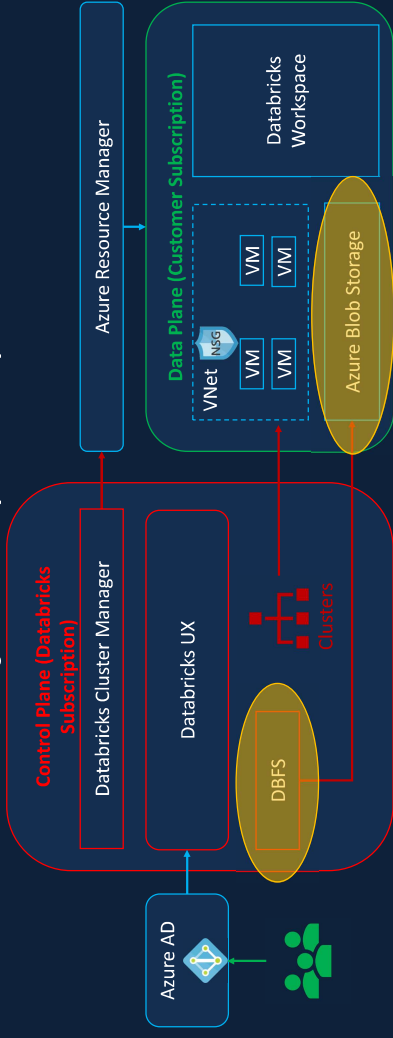


What is Databricks File System (DBFS)

What are Databricks mounts

Mount ADLS container to Databricks

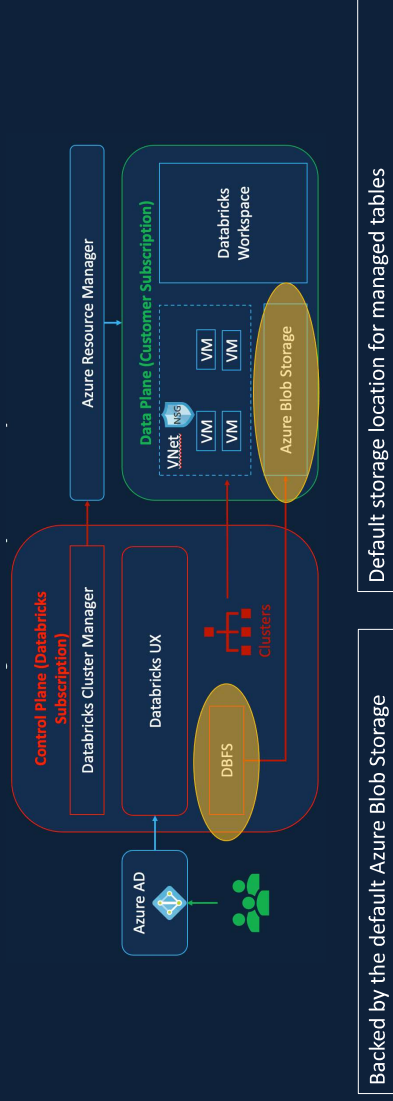
Databricks File System (DBFS)



Databricks File System (DBFS) is a distributed file system mounted on the Databricks workspace

DBFS Root is the default storage for a Databricks workspace created during workspace deployment

DBFS Root



Backed by the default Azure Blob Storage

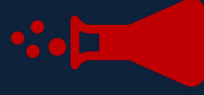
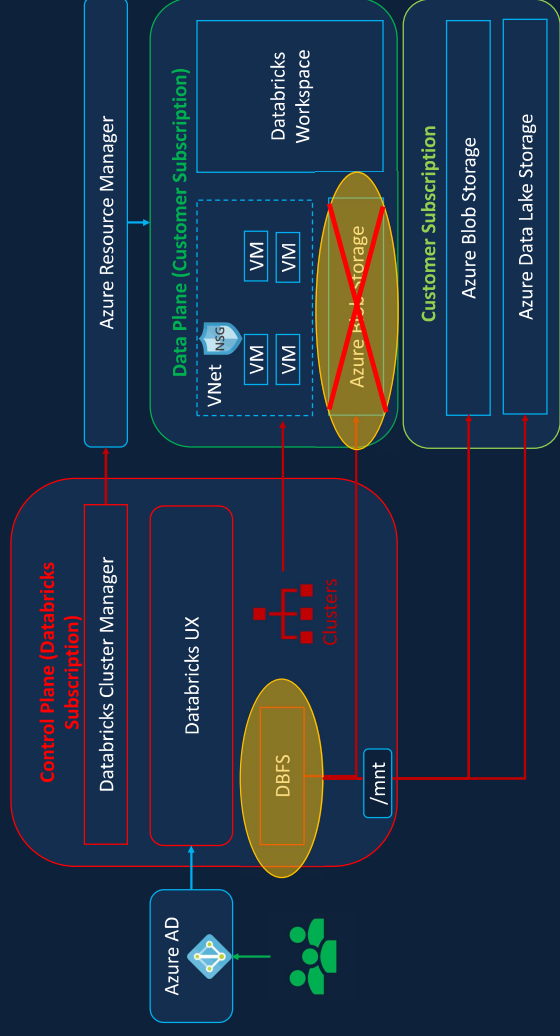
Default storage location for managed tables

Can be accessed via the web UI

Default storage location, but not recommended for user data

Query results are stored here

Databricks Mounts



DBFS Root Demo

Databricks Mounts

Benefits

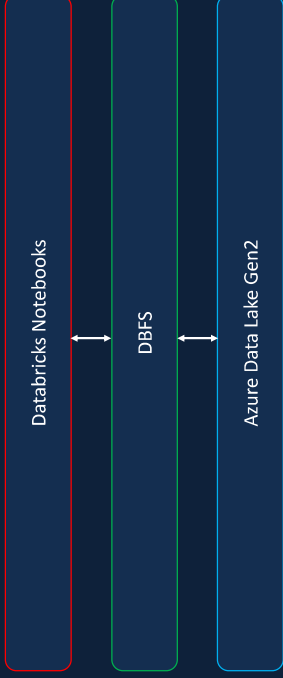
Access data without requiring credentials

Access files using file semantics rather than storage URLs (e.g. /mnt/storage1)

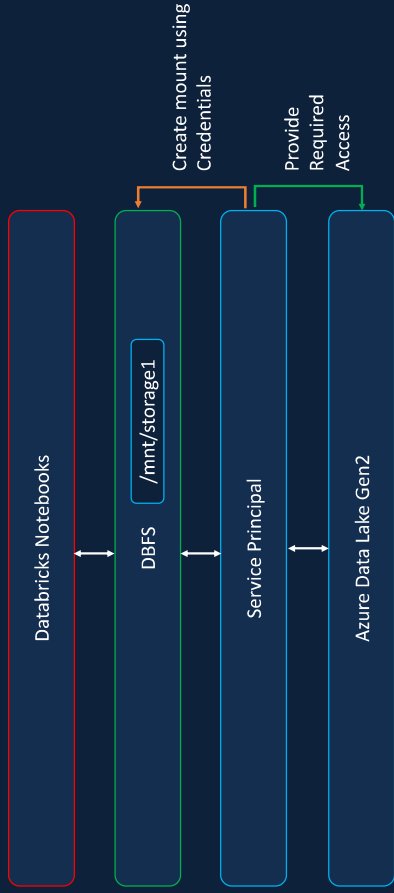
Stores files to object storage (e.g. Azure Blob), so you get all the benefits from Azure

Recommended solution for access Azure Storage until the introduction of Unity Catalog (Generally Available from end of 2022)

Databricks Mounts



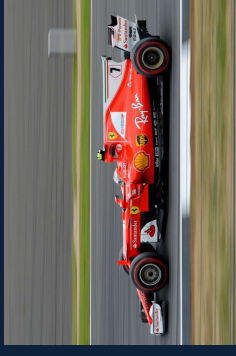
Databricks Mounts



Databricks Mounts



Project Overview



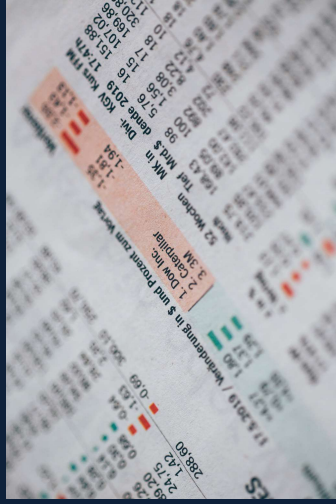
What is formula1

Formula1 data source & datasets

Prepare the data for the project

Project Requirements

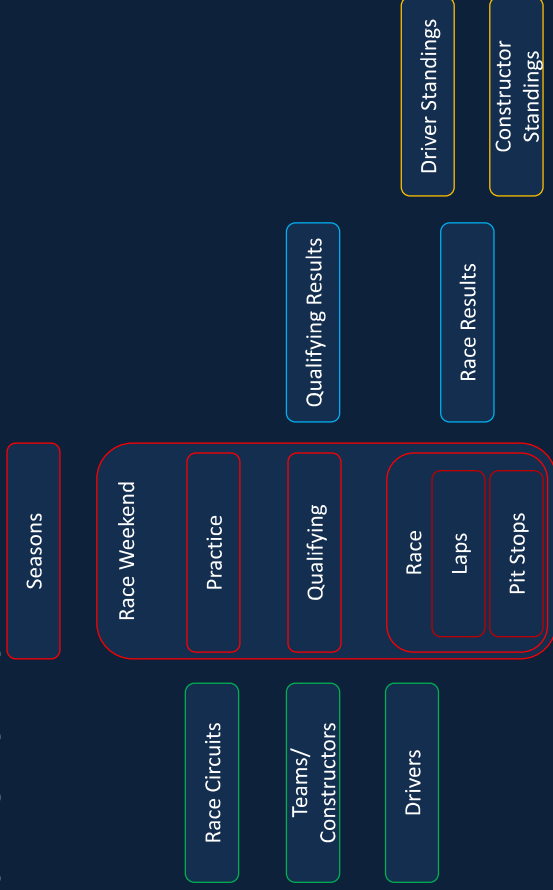
Data Overview



Formula1 Overview

Formula1 Data Source

<http://ergast.com/mrd/>



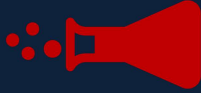
Formula1 Data Source

Formula1 Data Files



Circuits	CSV
Races	CSV
Constructors	Single Line JSON
Drivers	Single Line Nested JSON
Results	Single Line JSON
PitStops	Multi Line JSON
LapTimes	Split CSV Files
Qualifying	Split Multi Line JSON Files

Import Raw Data to Data Lake



Project Requirements

Data Ingestion Requirements



Ingest All 8 files into the data lake

Ingested data must have the schema applied

Ingested data must have audit columns

Ingested data must be stored in columnar format (i.e., Parquet)

Must be able to analyze the ingested data via SQL

Ingestion logic must be able to handle incremental load

Data Transformation Requirements



Join the key information required for reporting to create a new table.

Join the key information required for Analysis to create a new table.

Transformed tables must have audit columns

Must be able to analyze the transformed data via SQL

Transformed data must be stored in columnar format (i.e., Parquet)

Transformation logic must be able to handle incremental load

Reporting Requirements



Driver Standings

Constructor Standings

Analysis Requirements



Dominant Drivers

Dominant Teams

Visualize the outputs

Create Databricks Dashboards

Scheduling Requirements



Scheduled to run every Sunday 10PM

Ability to monitor pipelines

Ability to re-run failed pipelines

Ability to set-up alerts on failures

Other Non-Functional Requirements



Ability to delete individual records

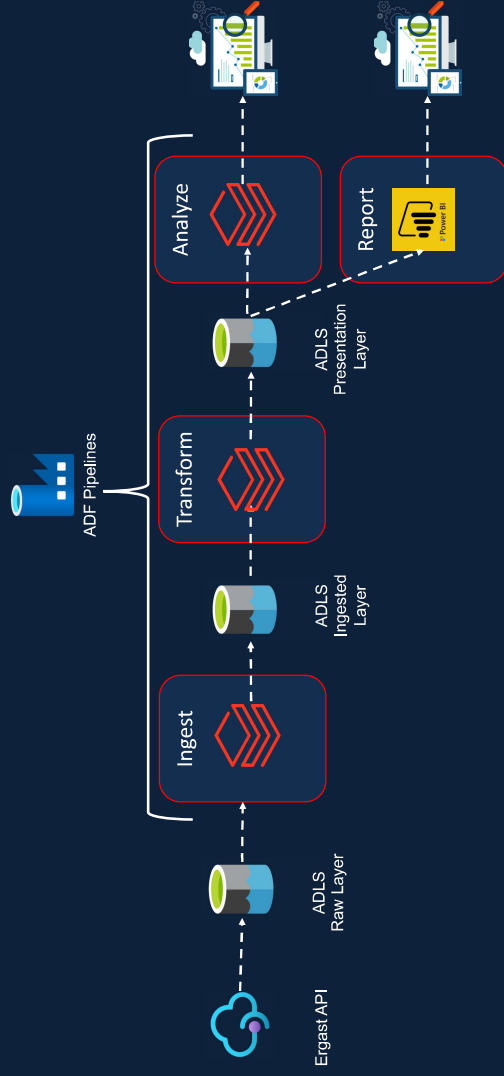
Ability to see history and time travel

Ability to roll back to a previous version

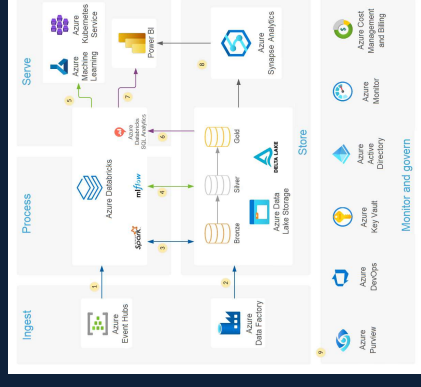
Blank

Solution Architecture Overview

Solution Architecture

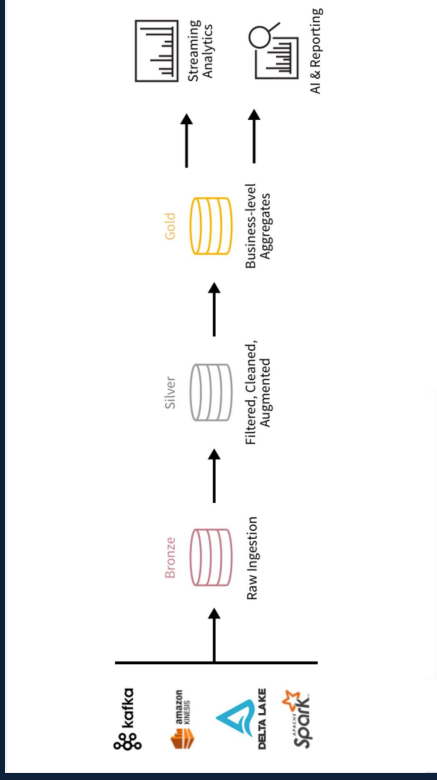


Azure Databricks Modern Analytics Architecture



<https://docs.microsoft.com/en-us/azure/architecture/solution-ideas/articles/azure-databricks-modern-analytics-architecture>

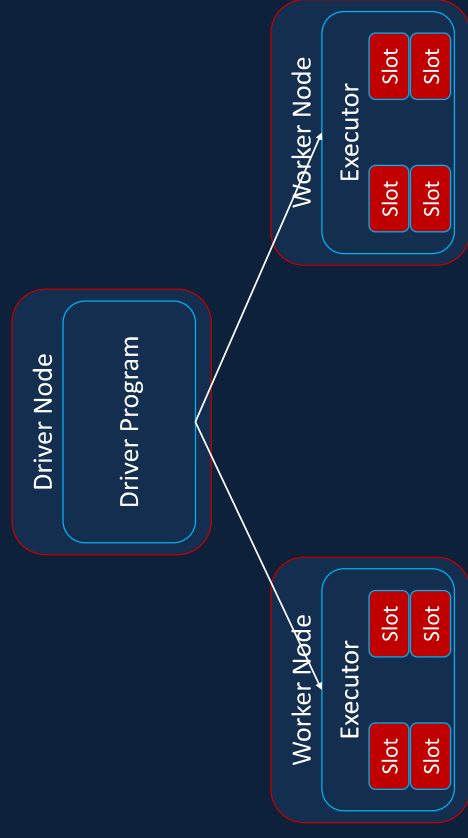
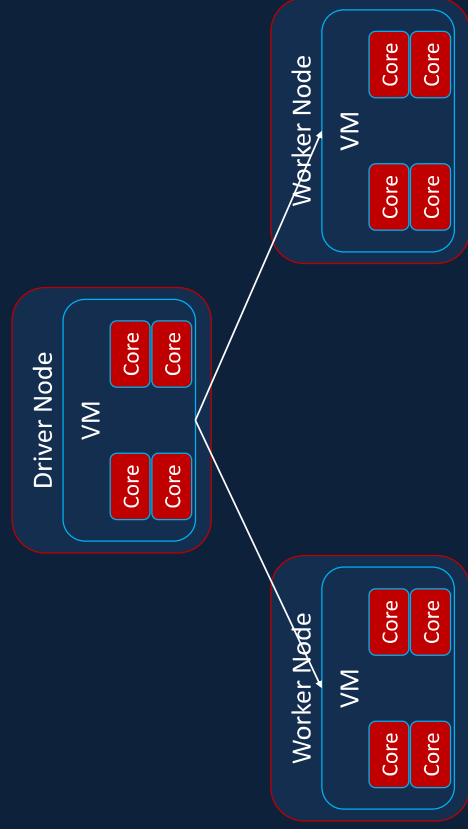
Databricks Architecture



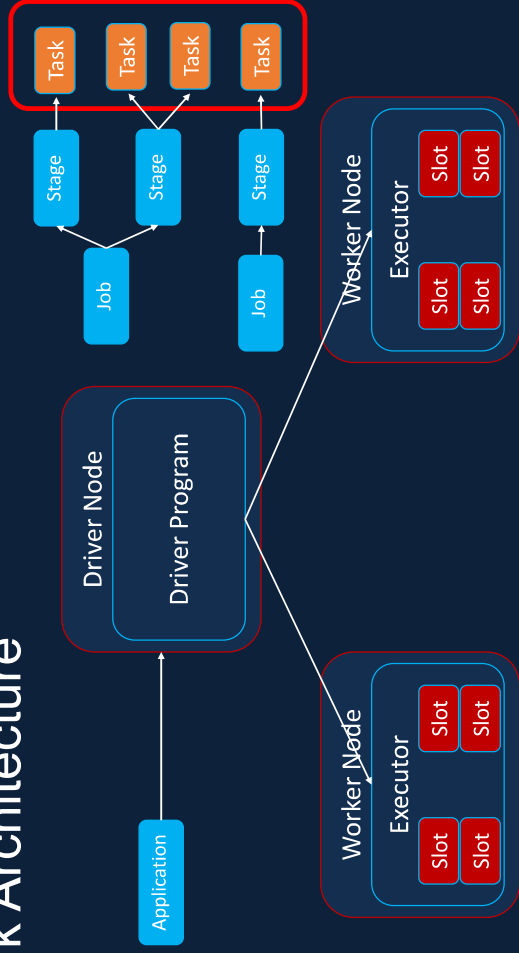
Spark Architecture

<https://databricks.com/solutions/data-pipelines>

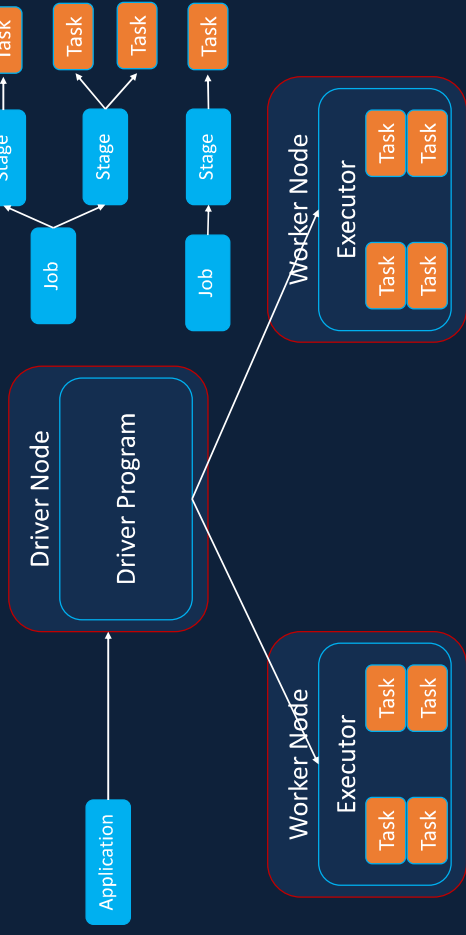
Spark Architecture



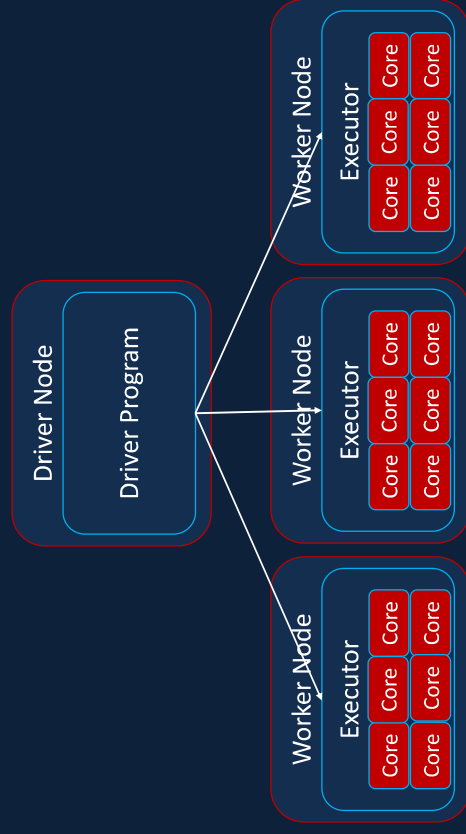
Spark Architecture



Spark Architecture



Spark Architecture – Cluster Scaling



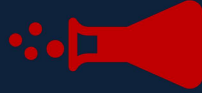
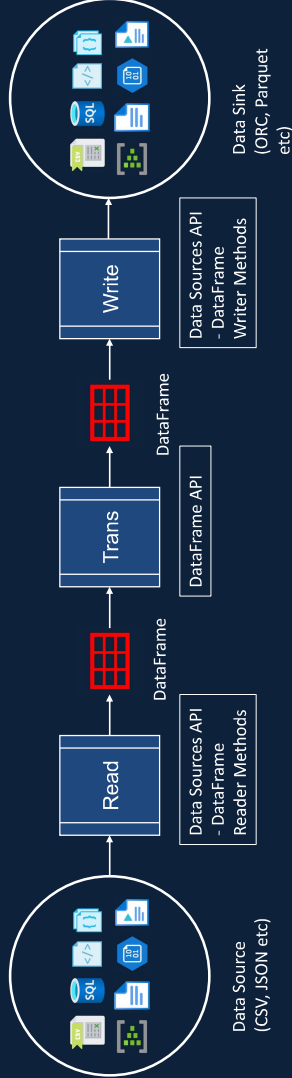
Spark DataFrame

Spark DataFrame

Spark DataFrame

	Driver	Team	Wins	Points
1	Max Verstappen	Red Bull	5	182
2	Lewis Hamilton	Mercedes	3	150
3	Sergio Perez	Red Bull	1	104
4	Lando Norris	McLaren	0	101
5	Valtteri Bottas	Mercedes	0	92
6	Charles Leclerc	Ferrari	0	62
7	Carlos Sainz Jr.	Ferrari	0	60
8	Daniel Ricciardo	McLaren	0	40
9	Pierre Gasly	AlphaTauri	0	39
10	Sebastian Vettel	Aston Martin	0	30
11	Fernando Alonso	Alpine	0	20
12	Lance Stroll	Aston Martin	0	14

Source: <https://www.bbc.co.uk/sport/formula1/drivers-world-championship/standings>



Spark Documentation

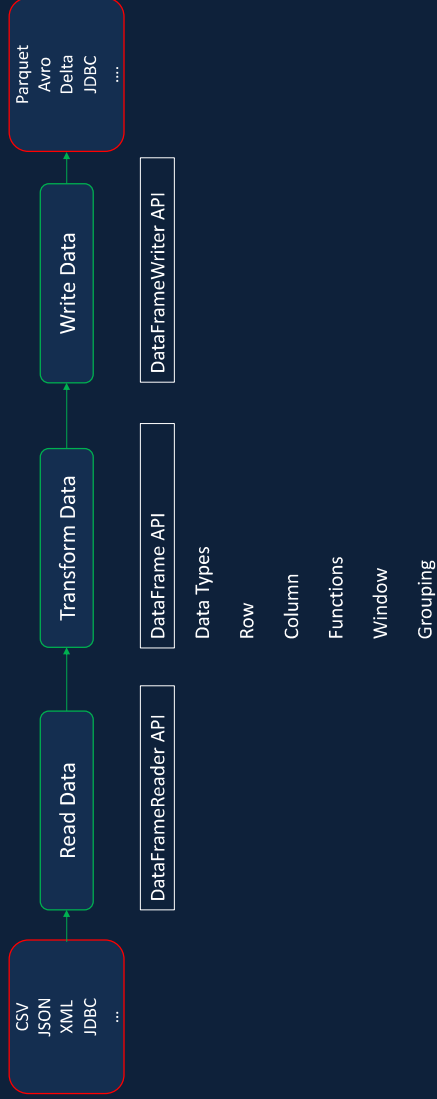
Data Ingestion Overview

Data Ingestion Requirements



- Ingest All 8 files into the data lake
- Ingested data must have the schema applied
- Ingested data must have audit columns
- Ingested data must be stored in columnar format (i.e., Parquet)
- Must be able to analyze the ingested data via SQL
- Ingestion logic must be able to handle incremental load

Data Ingestion Overview



Data Ingestion Overview

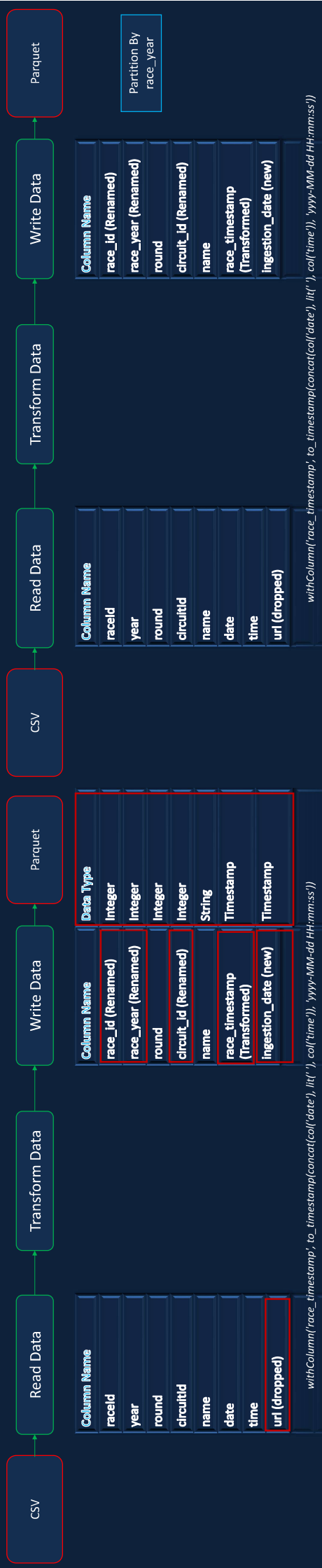
File Name	File Type	Assignment/ Class work
Circuits	CSV	Class work
Races	CSV	Assignment
Constructors	Single Line JSON	Class work
Results	Single Line JSON	Assignment
Drivers	Single Line Nested JSON	Class work
PitStops	Multi Line JSON	Class work
LapTimes	Split CSV Files	Class work
Qualifying	Split Multi Line JSON Files	Assignment

Data Ingestion - Circuits



Data Ingestion – Races (Assignment)

Data Ingestion – Races (Partition By)



Data Ingestion - Constructors

Data Ingestion - Drivers



Data Ingestion – Results (Assignment)



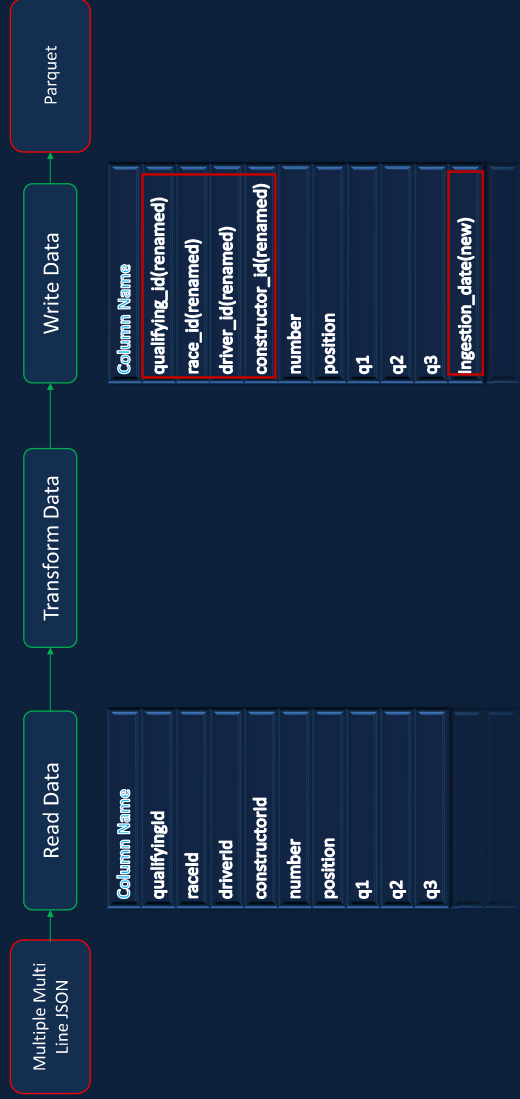
Data Ingestion - Pitstops



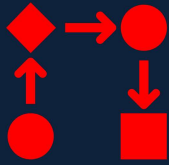
Data Ingestion - Laptimes



Data Ingestion – Qualifying (Assignment)

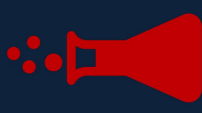


Databricks Workflows

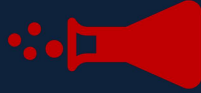


- Include notebook
- Defining notebook parameters
- Notebook workflow
- Databricks Jobs

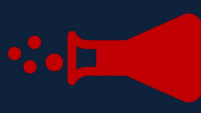
Include notebook (%run)



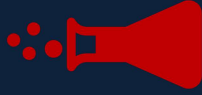
Passing Parameters (widgets)



Notebook Workflow



Databricks Jobs



Filter/ Join Transformations



Filter Transformation

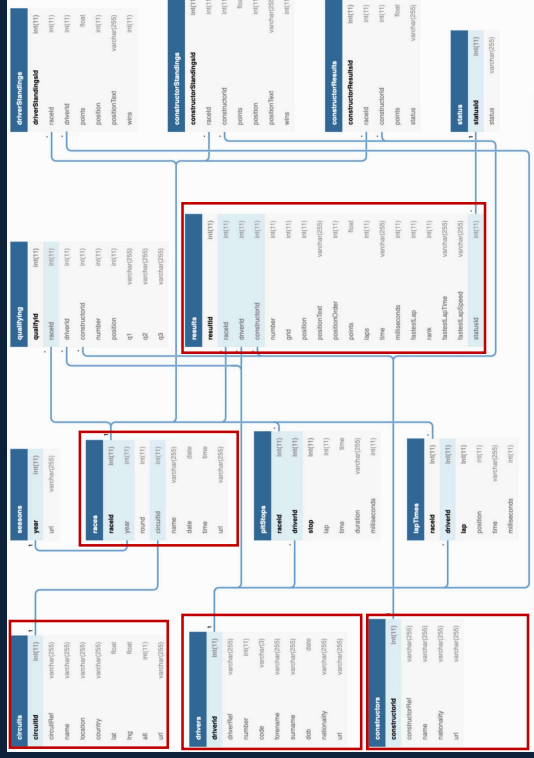
Join Transformations

Apply Transformations to F1 Project

Filter Transformations

Join Transformations

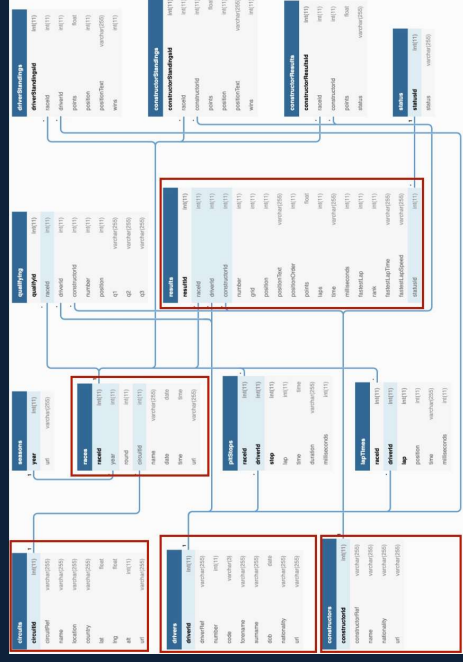
Join -Race Results



Join Transformation

Race Results

Join -Race Results

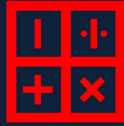


Column Name	Source
race_year	races
race_name	races
race_date	races
circuit_location	circuits
driver_name	drivers
driver_number	drivers
driver_nationality	drivers
team	constructors
grid	results
fastest_lap	results
race_time	results
points	results
created_date	current_timestamp



Set-up Environment Presentation Layer

Aggregations



Simple Aggregations

Grouped Aggregations

Window Functions

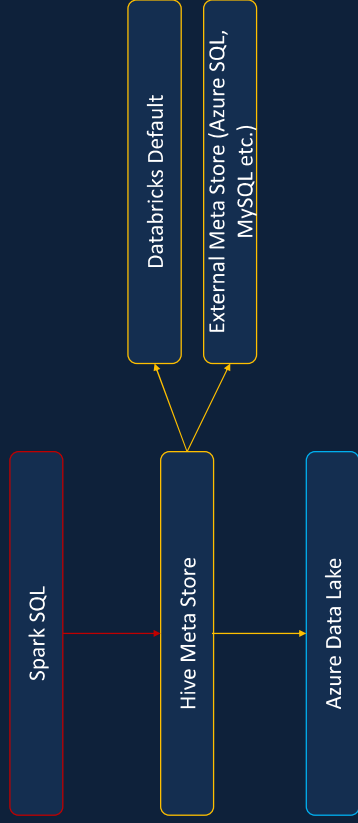
Apply Aggregations to F1 Project

Built-in Aggregate Functions

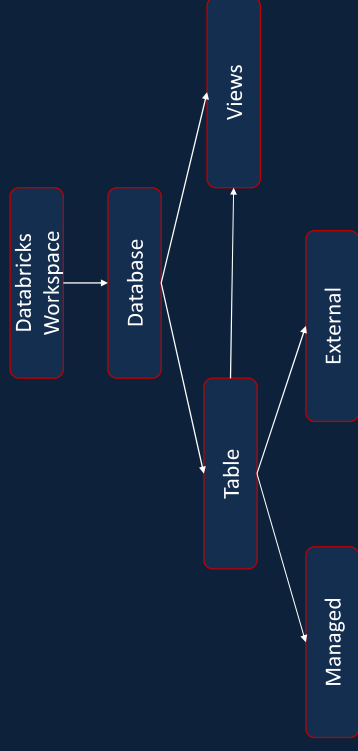
Group By

Databases/ Tables/ Views

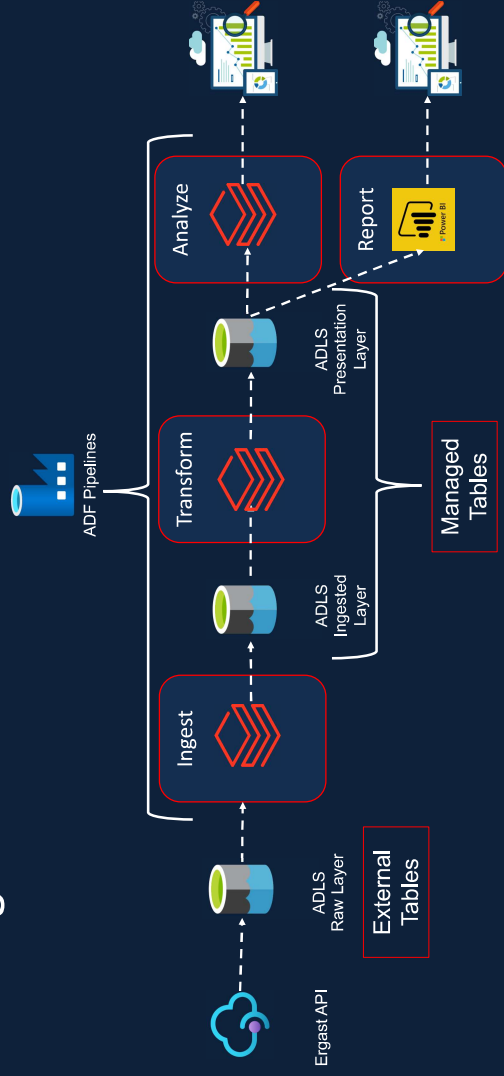
Hive Meta Store



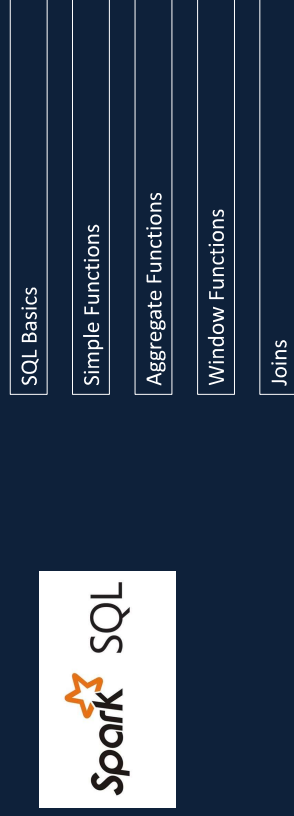
Spark Databases/ Tables/ Views



Managed/ External Tables



Spark SQL Introduction



Dominant Drivers/ Teams Analysis

Create a table with the data required

Granularity of the data – race_year, driver, team

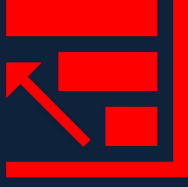
Rank the dominant drivers of all time/ last decade etc

Rank the dominant teams of all time/ last decade etc

Visualization of dominant drivers

Visualization of dominant teams

Incremental Load



Data Loading Patterns

F1 Project Load Pattern

Implementation

Data Load Types

Full Load

Incremental Load

Full Dataset

Day 1

Race 1

Day 2

Race 1

Race 2

Day 3

Race 1

Race 2

Race 3

Day 4

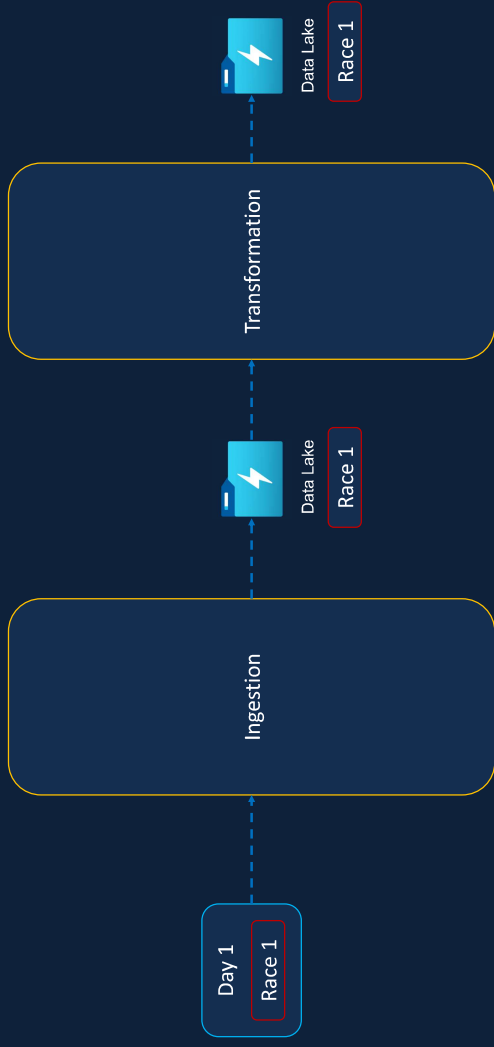
Race 1

Race 2

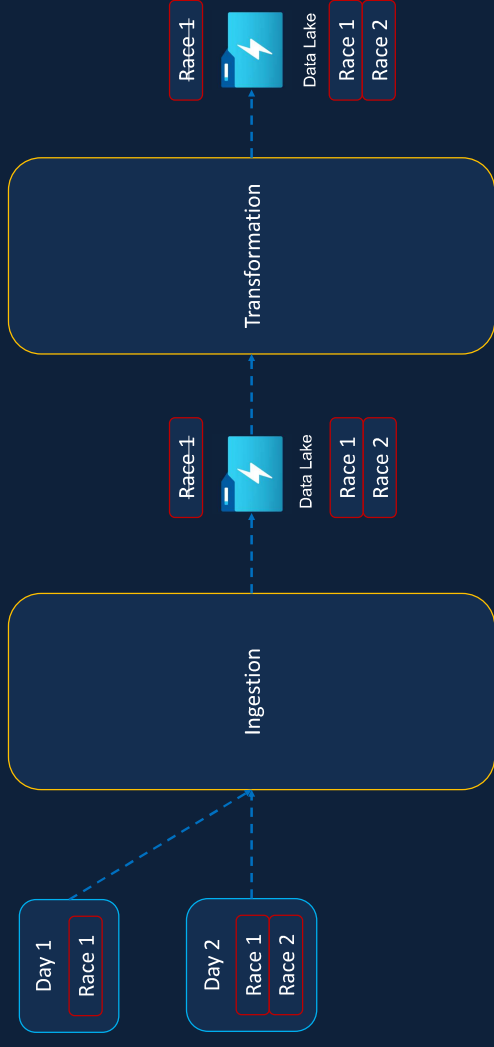
Race 3

Race 4

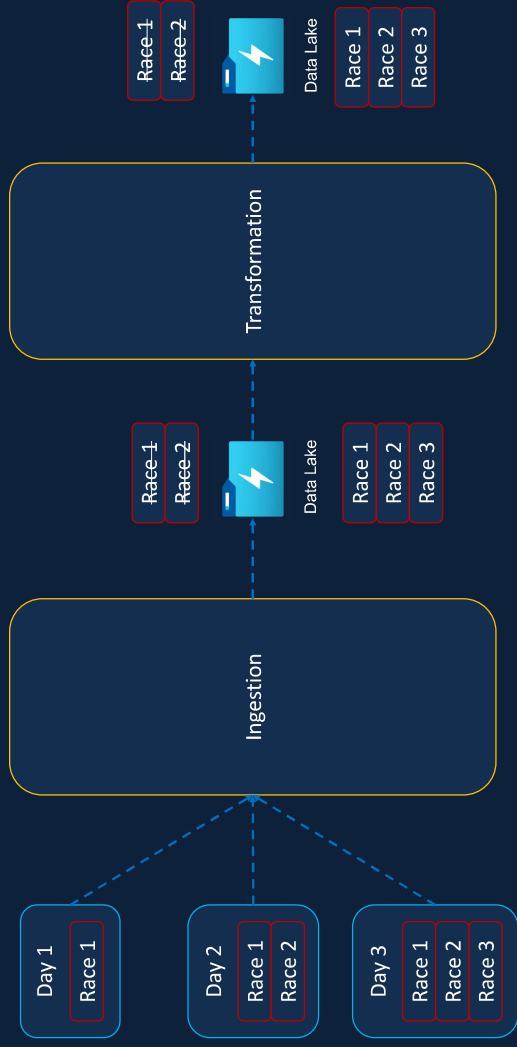
Full Refresh/ Load – Day 1



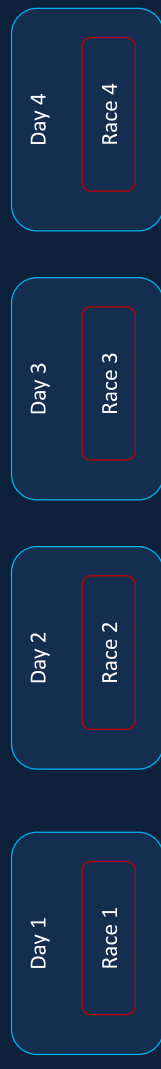
Full Refresh/ Load – Day 2



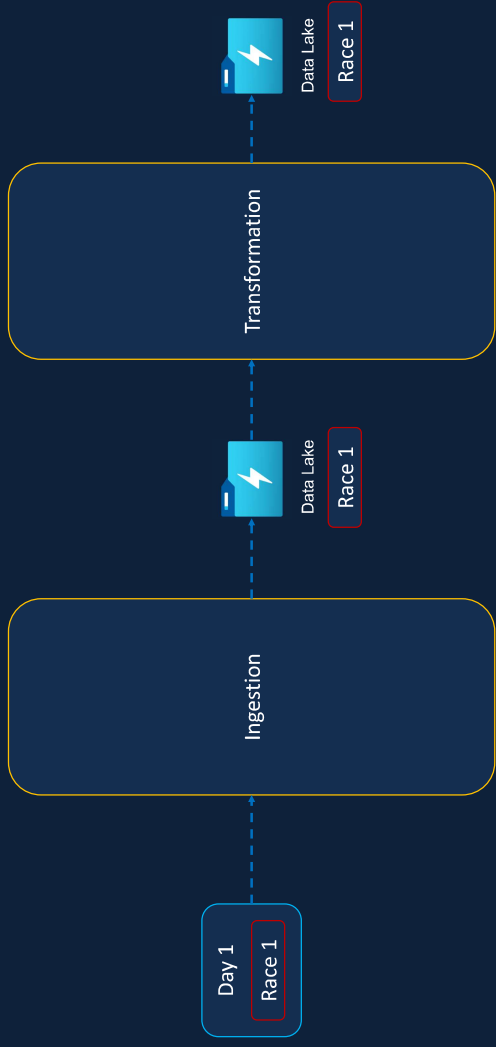
Full Refresh/ Load – Day 3



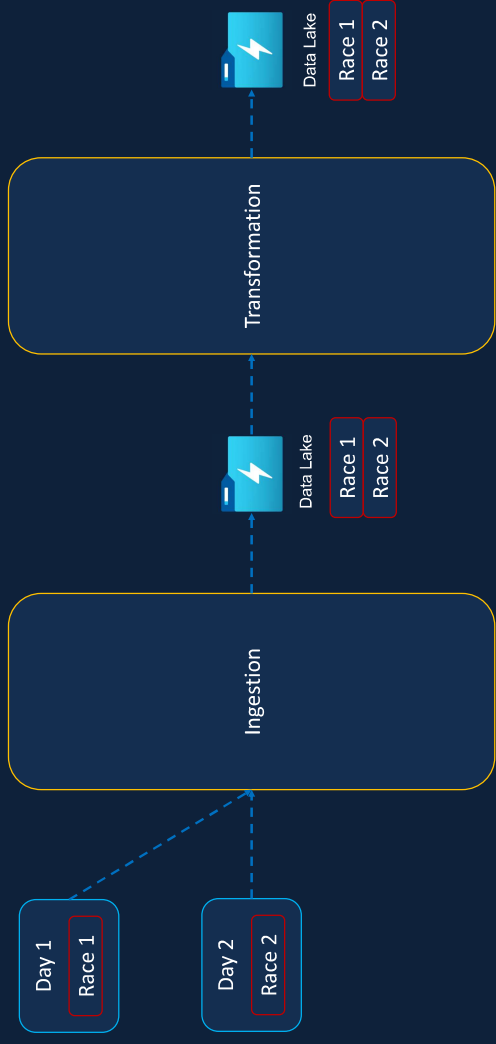
Incremental Dataset



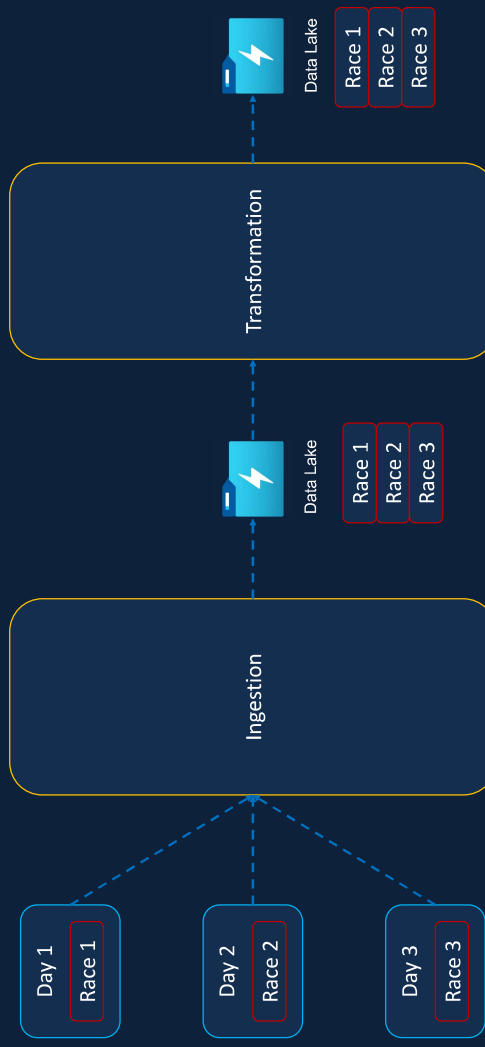
Incremental Load – Day 1



Incremental Load – Day 2



Incremental Load – Day 3



Hybrid Scenarios

Full Dataset received, but data loaded & transformed incrementally

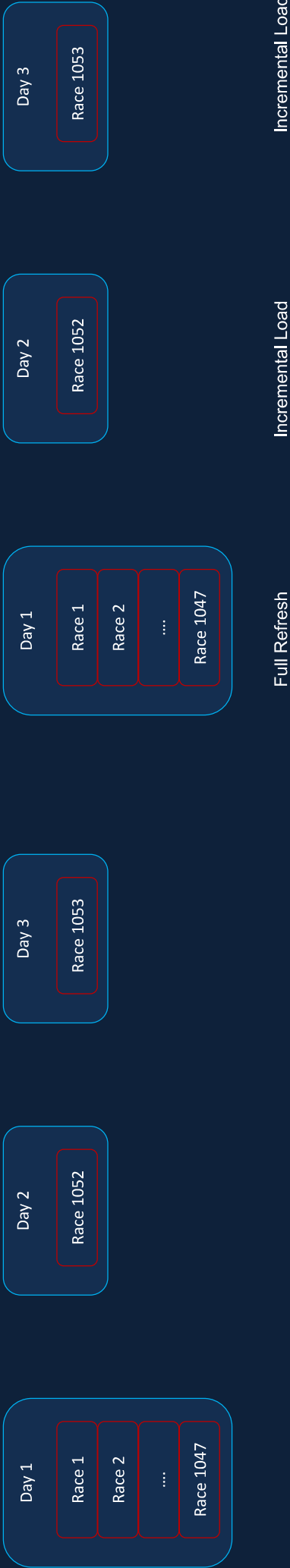
Incremental dataset received, but data loaded & transformed in full

Data received contains both full and incremental files

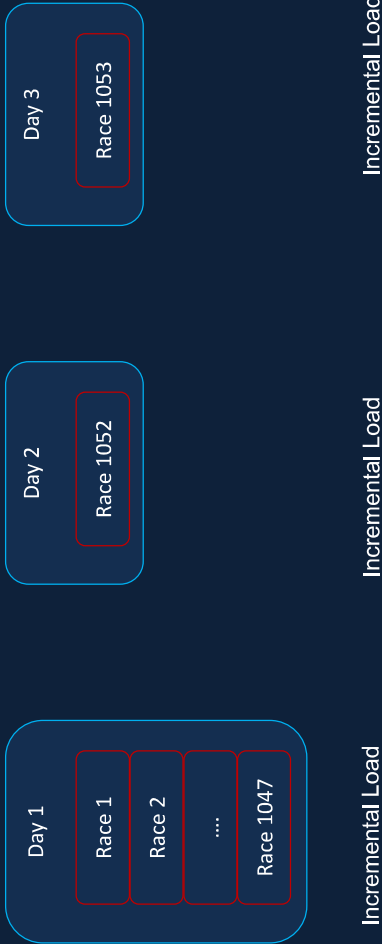
Incremental data received. Ingested incrementally & transformed in full

Formula1 Scenario

Formula1 Scenario / Solution 1



Formula1 Scenario / Solution 2

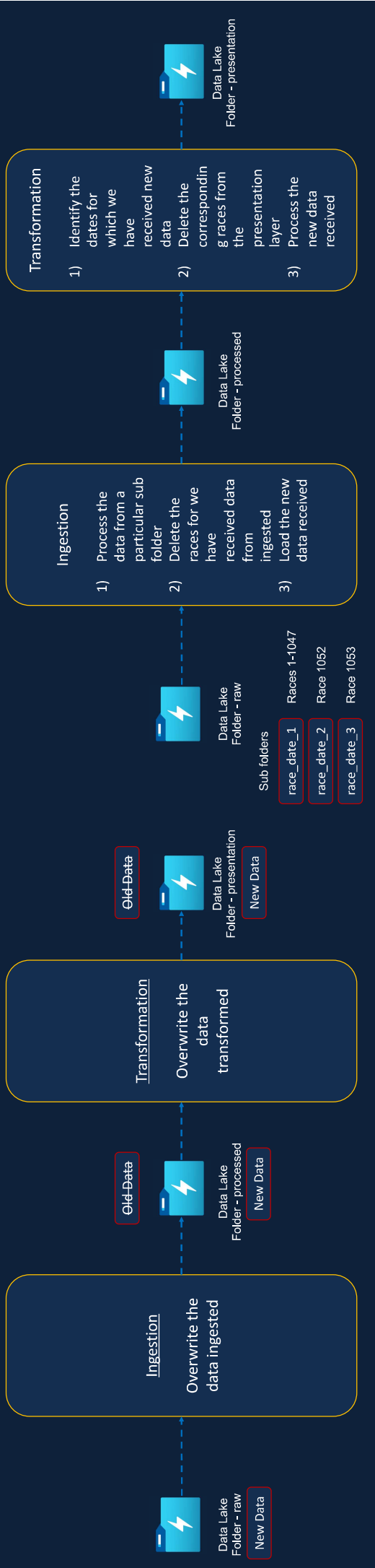


Formula1 Data Files

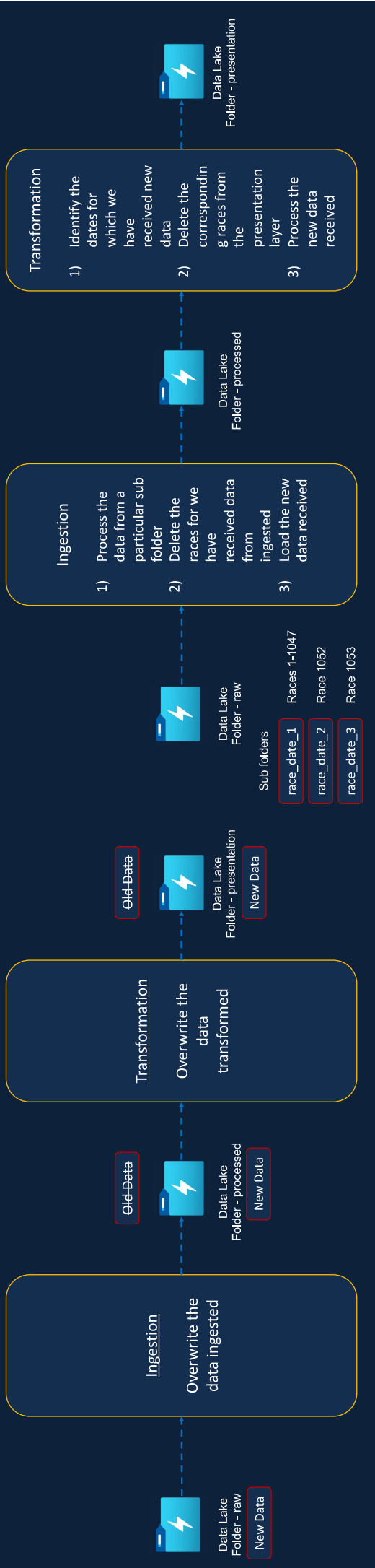


Circuits	Data from all races
Races	Data from all races
Constructors	Data from all races
Drivers	Data from all races
Results	Data only from that race
PitStops	Data only from that race
LapTimes	Data only from that race
Qualifying	Data only from that race

Current Solution



New Solution



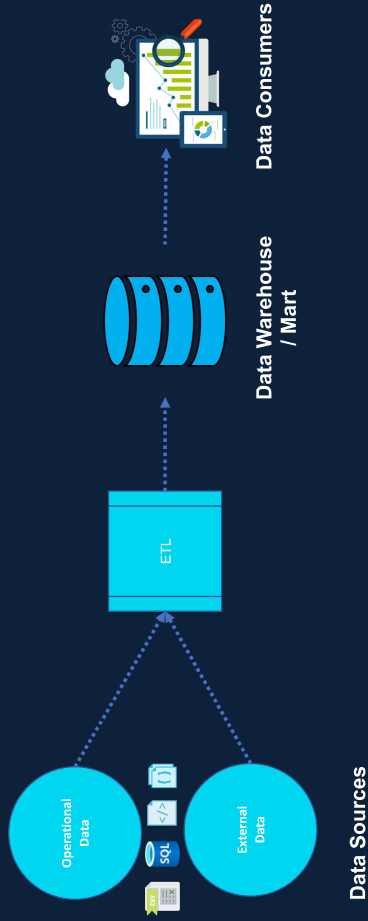
Delta Lake

- Pitfalls of Data Lakes
- Lakehouse Architecture
- Delta Lake Capabilities
- Convert F1 project to Delta Lake



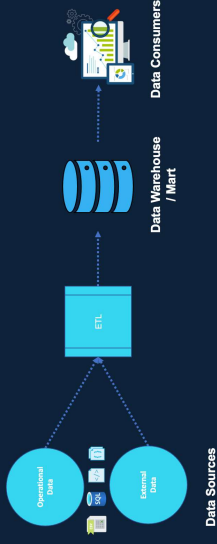
Delta Lake

Data Warehouse

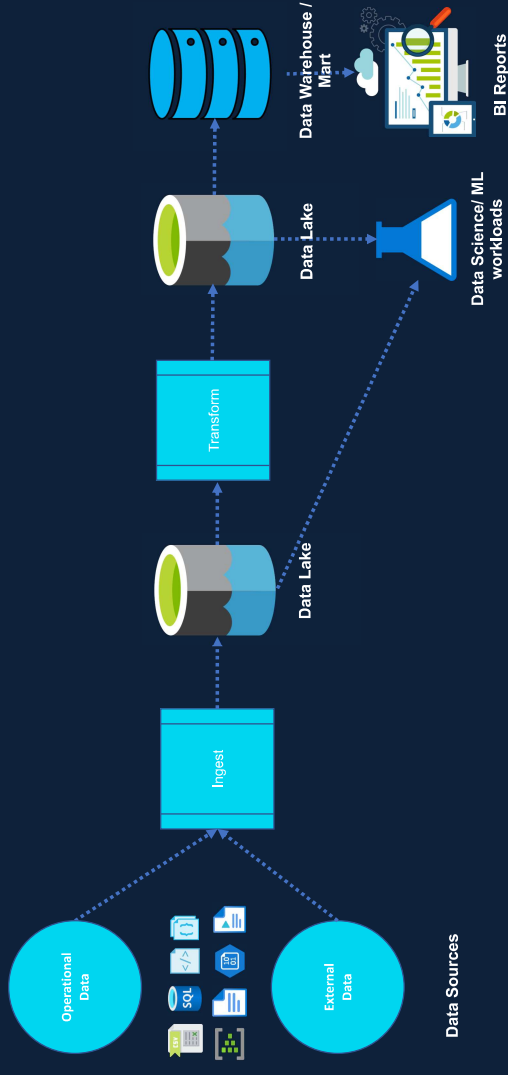


- Lack of support for unstructured data
- Longer to ingest new data
- Proprietary data formats
- Scalability
- Expensive to store data
- Lack of support for ML/ AI workloads

Data Warehouse

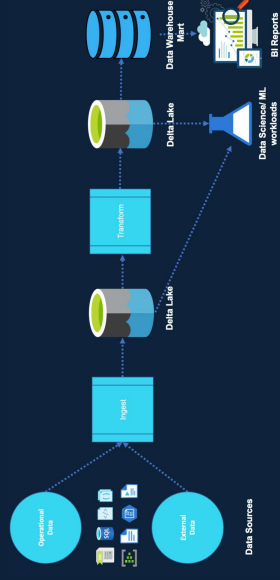


Data Lake

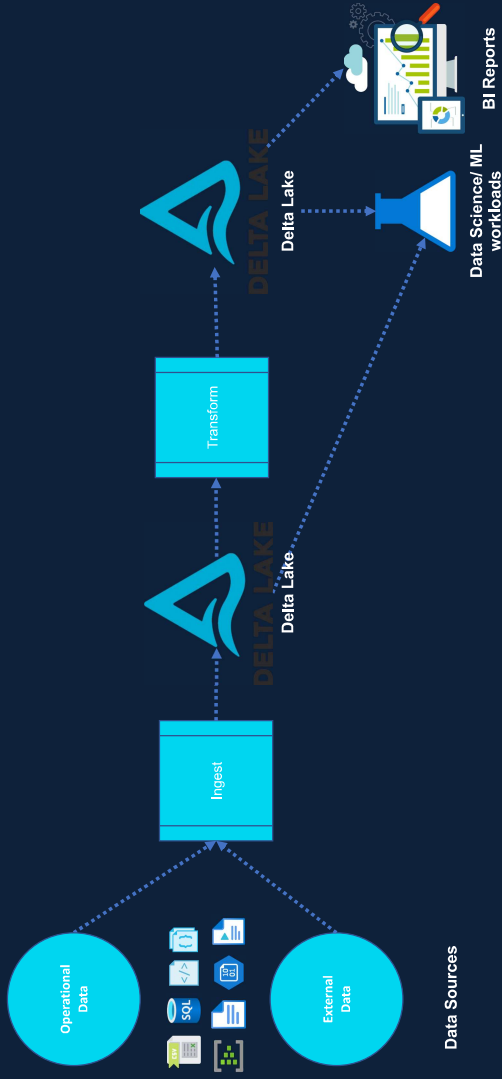


- No support for ACID transactions
- Failed jobs leave partial files
- Inconsistent reads
- Unable to handle corrections to data
- Unable to roll back any data.
- Lack of ability remove data for GDPR etc
- No history or versioning
- Poor performance
- Poor BI support
- Complex to set-up
- Lambda architecture for streaming workloads

Data Lake

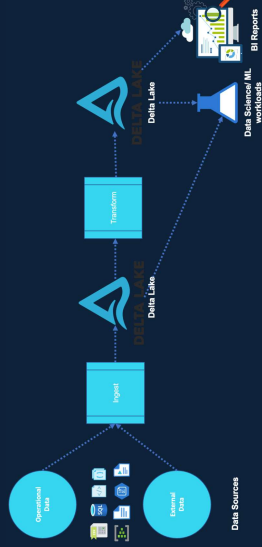


Data Lakehouse

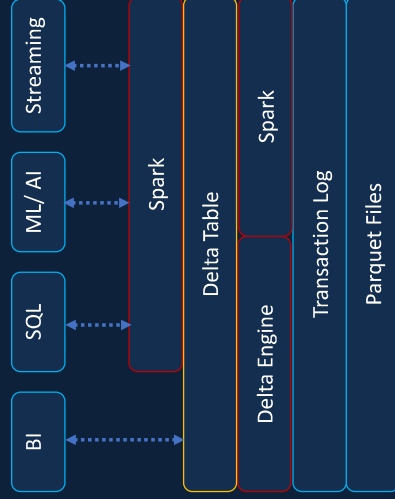


Data Lakehouse

- Handles all types of data
- Cheap cloud object storage
- Uses open source format
- Support for all types of workloads
- Ability to use BI tools directly
- ACID support
- History & Versioning
- Better performance
- Simple architecture



Delta Lake



Simple Architecture

Transformations, ML, Streaming etc.

Data security, Governance, Integrity, Perf.

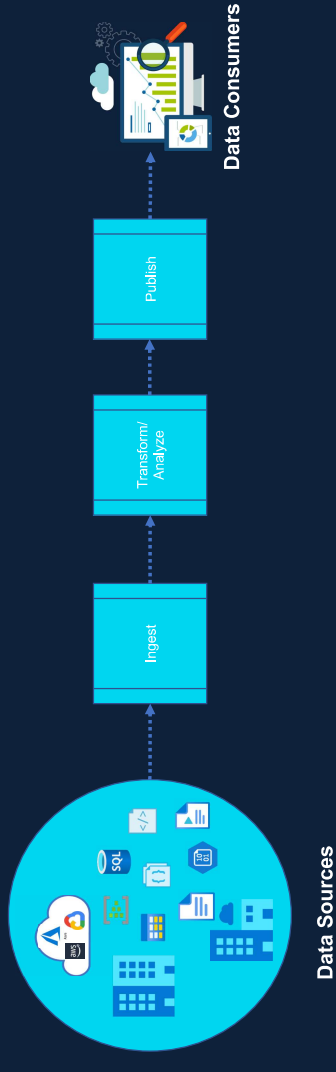
Optimized engine for performance

History, Versioning, ACID, Time Travel

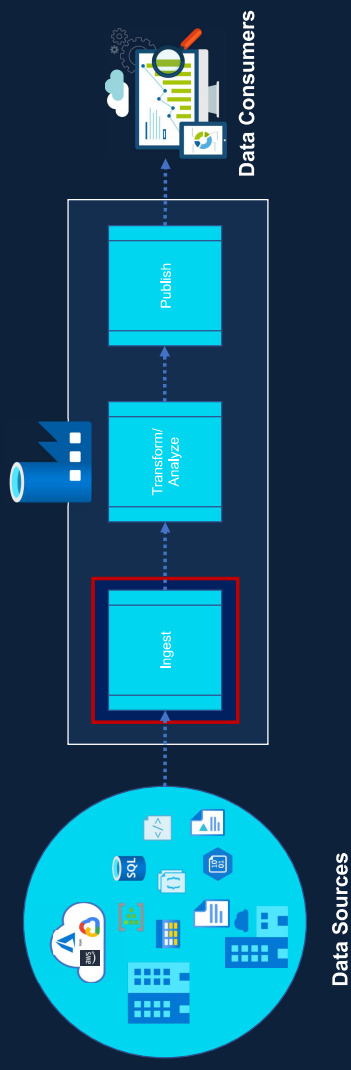
Open file format

Delta Lake Demo

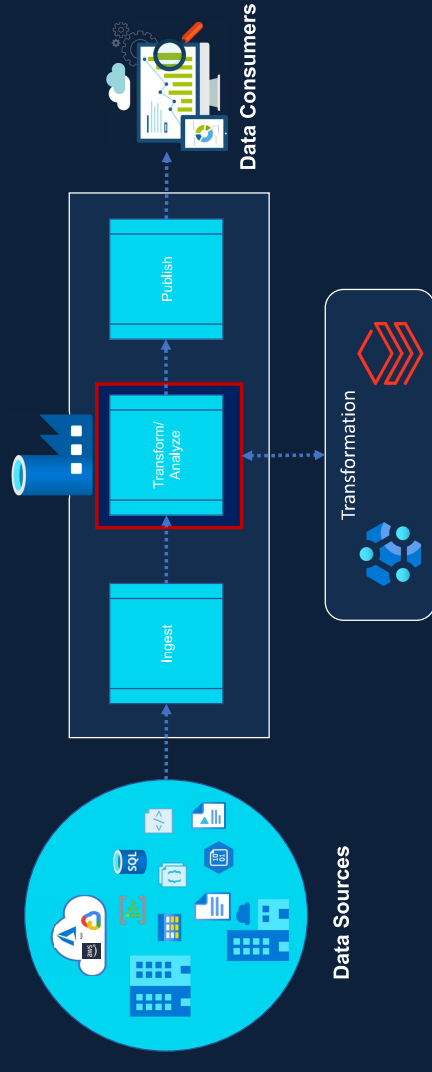
The Data Problem



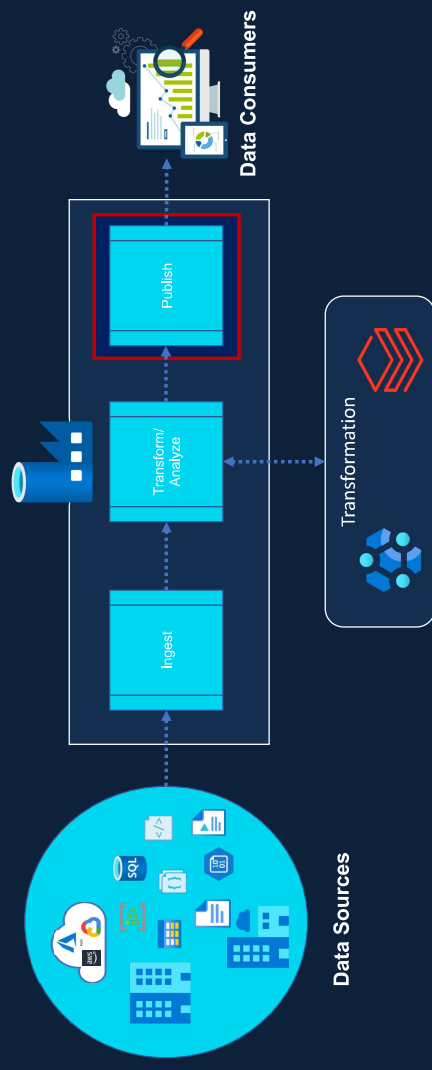
The Data Problem



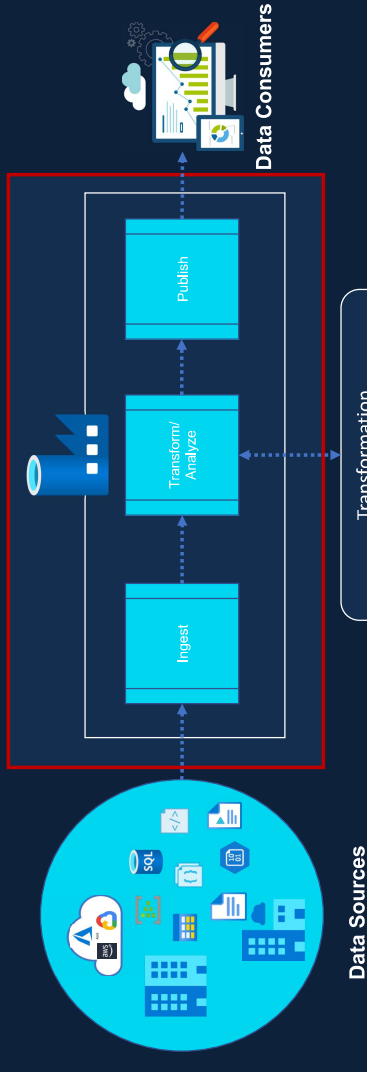
The Data Problem



The Data Problem



The Data Problem



What is Azure Data Factory



- | |
|-----------------------------|
| Fully Managed Service |
| |
| Serverless |
| |
| Data Integration Service |
| |
| Data Transformation Service |
| |
| Data Orchestration Service |

A fully managed, serverless data integration solution for ingesting, preparing and transforming all of your data at scale.

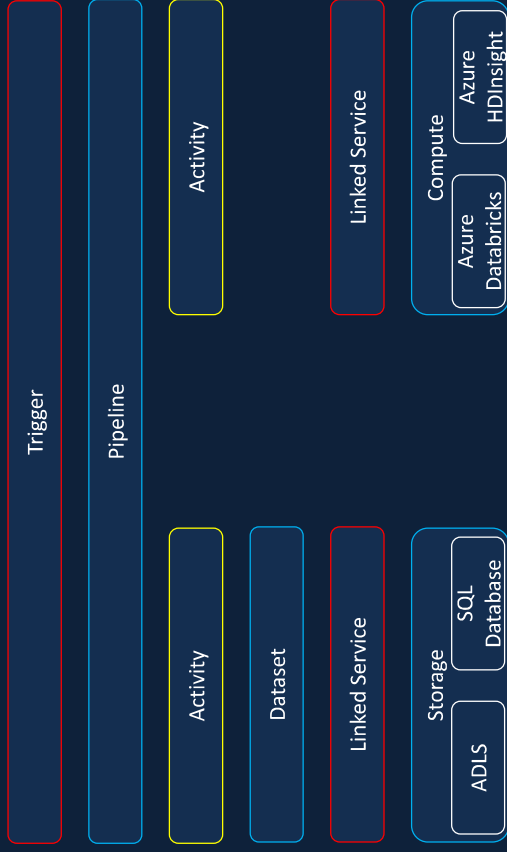
What Azure Data Factory Is Not



- | |
|---|
| Data Migration Tool |
| Data Streaming Service |
| Suitable for Complex Data Transformations |
| Data Storage Service |

Create Data Factory Service

Data Factory Components



Connecting from Power BI

Unity Catalog - Introduction



- Unity Catalog Overview
- Enable Unity Catalog Metastore
- Cluster Configurations
- Unity Catalog Object Model
- Access External Data Lake

Unity Catalog



Unity Catalog



Unity Catalog is a Databricks offered unified solution for implementing data governance in the Data Lakehouse



Unity Catalog is a Databricks offered unified solution for implementing data governance in the Data Lakehouse

Unity Catalog

Data Governance

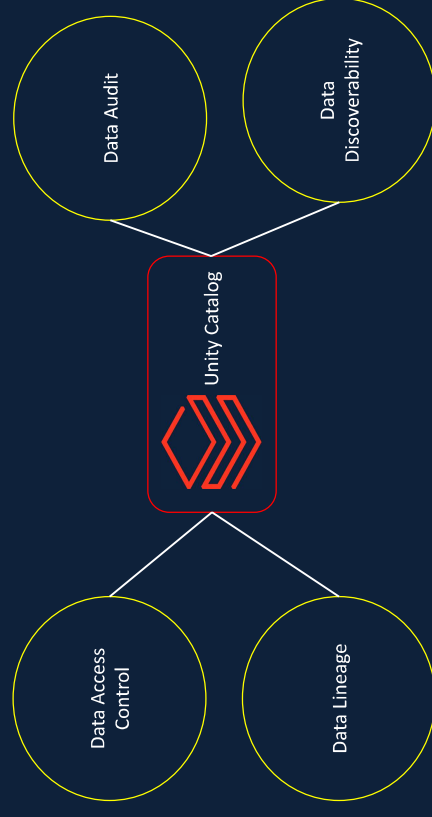
Is the process of managing the availability, usability, integrity and security of the data present in an enterprise

Controls access to the data for the users

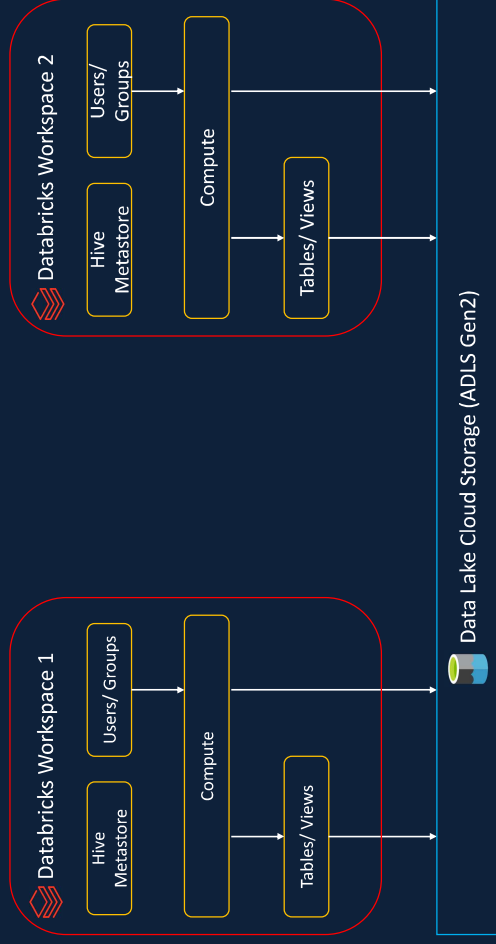
Ensures that the data is trust worthy and not misused

Helps implement privacy regulations such as GDPR, CCPA etc

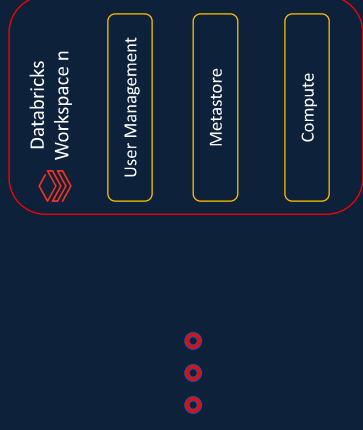
Data Governance



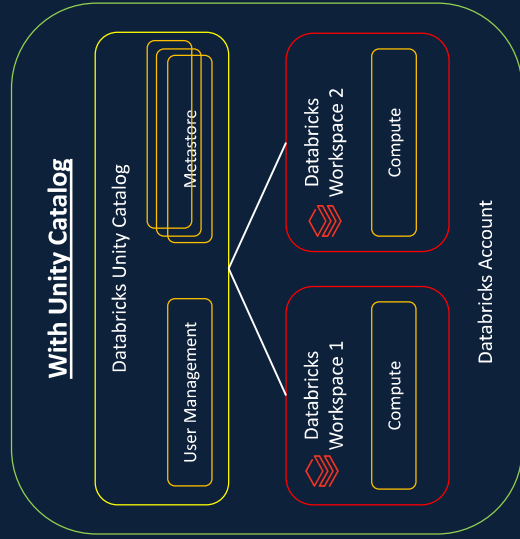
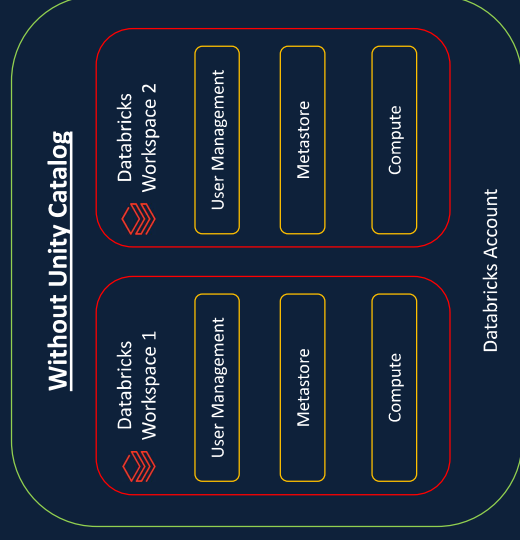
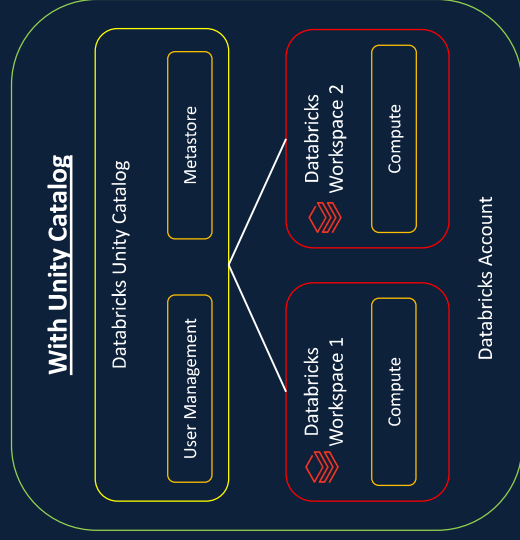
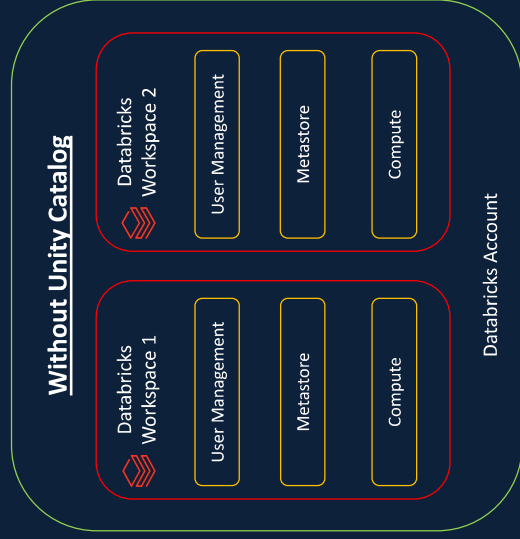
Without Unity Catalog



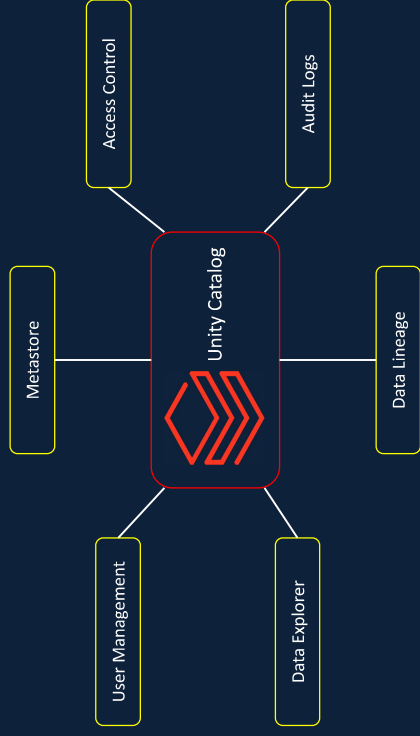
Without Unity Catalog



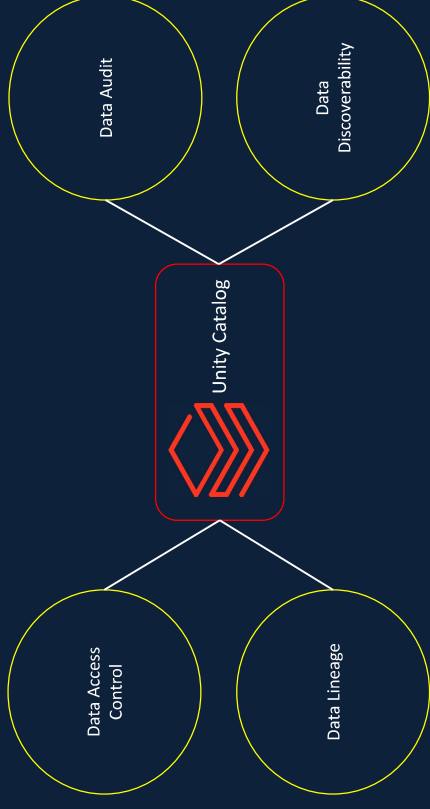
With Unity Catalog



Unity Catalog

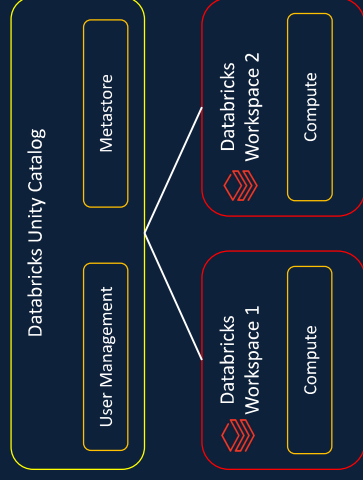


Unity Catalog

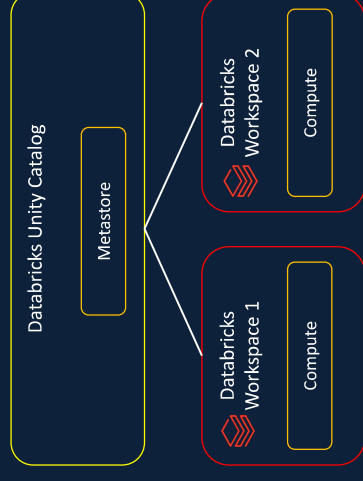


Unity Catalog is a Databricks offered unified solution for implementing data governance in the Data Lakehouse

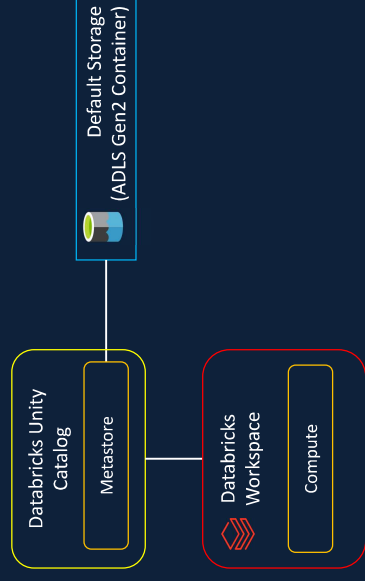
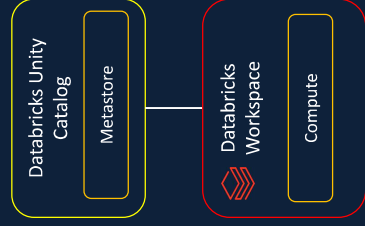
Unity Catalog Set-up



Unity Catalog Set-up

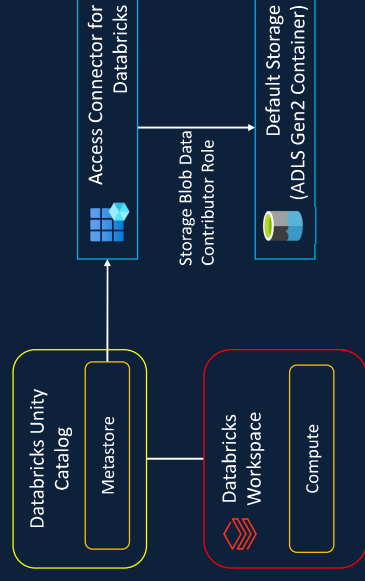
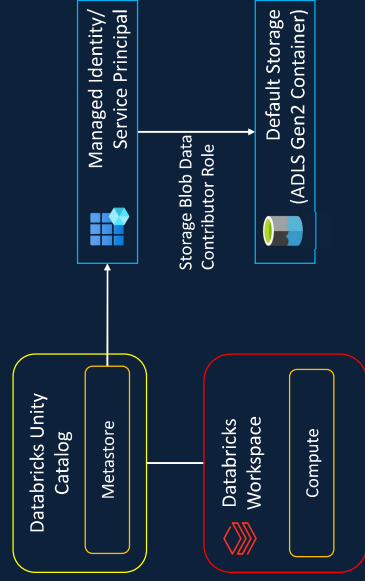


Unity Catalog Set-up

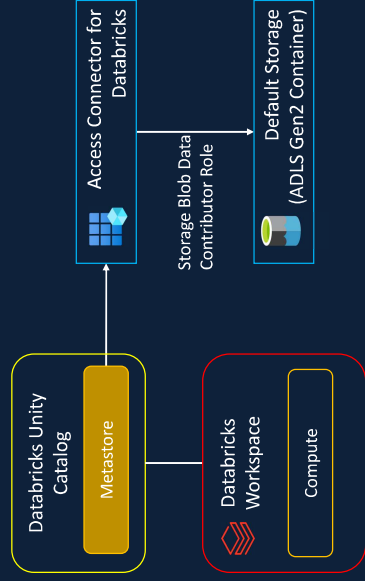


Unity Catalog Set-up

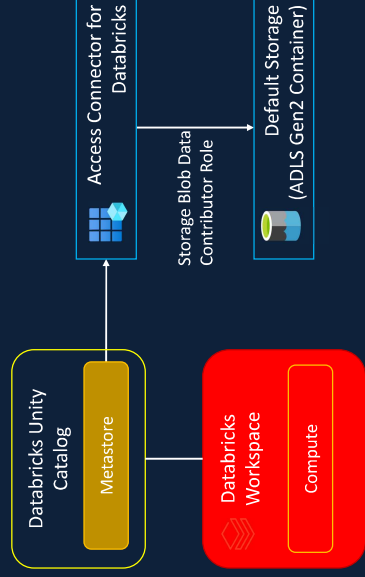
Unity Catalog Set-up



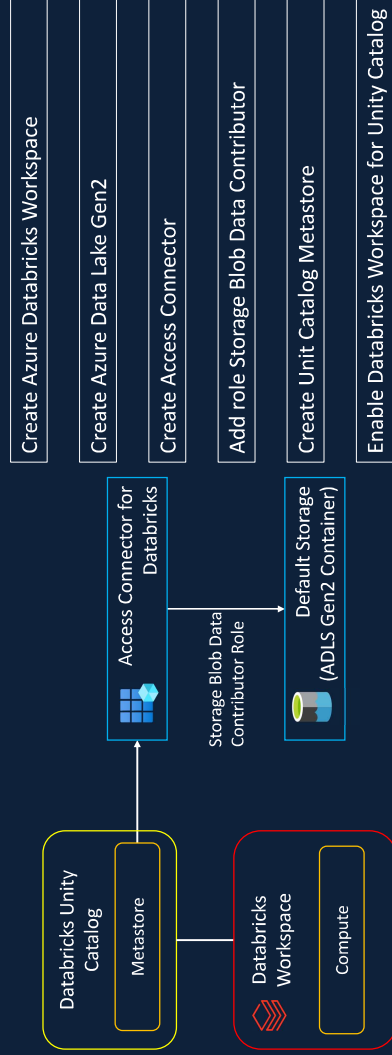
Unity Catalog Set-up



Unity Catalog Set-up



Unity Catalog Set-up

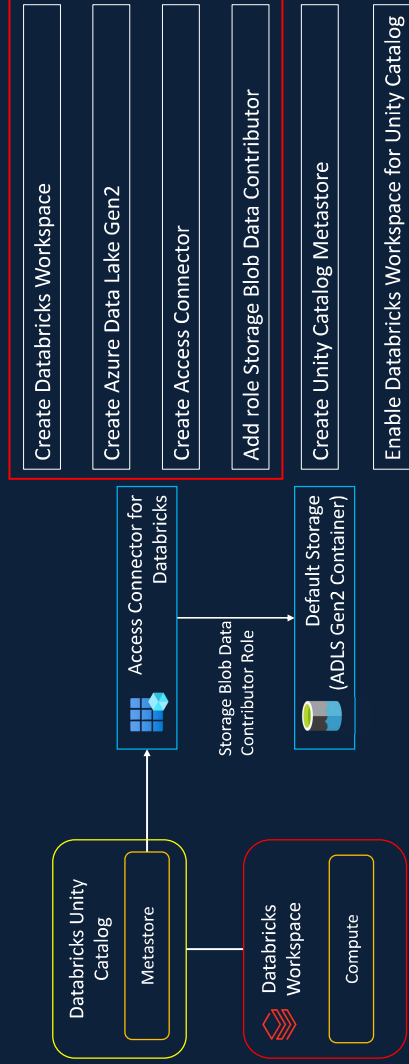


Unity Catalog Set-up

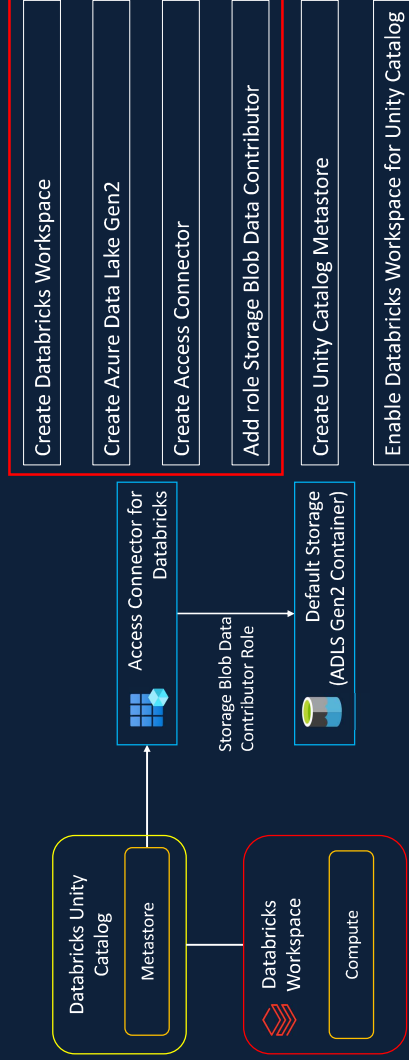
Pre-requisites



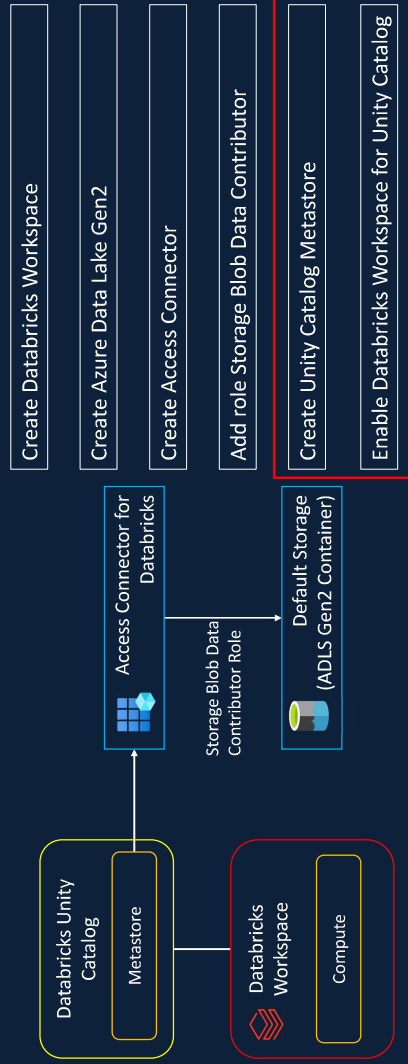
Unity Catalog Set-up



Unity Catalog Set-up



Unity Catalog Set-up



Cluster Configuration



Unity Catalog Object Model



Top level container

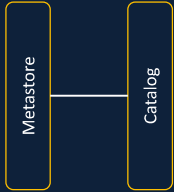
Only one per region

Paired with Default ADLS Storage

Unity Catalog Object Model

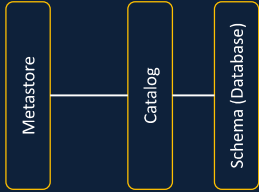


Unity Catalog Object Model



Newly added to Unity Catalog

Logical container within Metastore

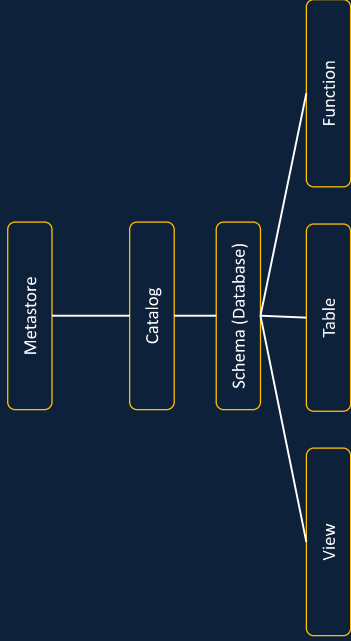


Next level container within Catalogs

Schemas and Databases are the same

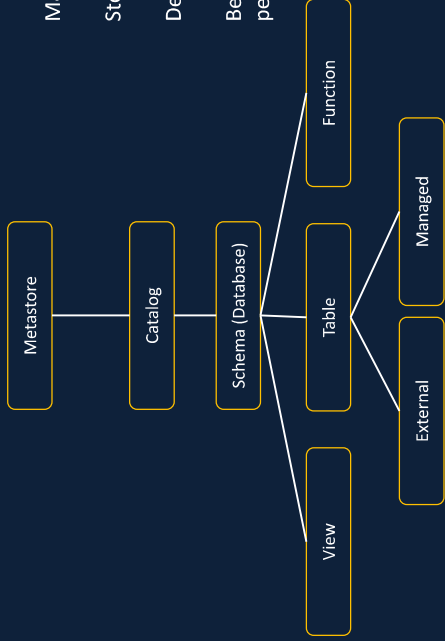
Use Schema instead of Database

Unity Catalog Object Model

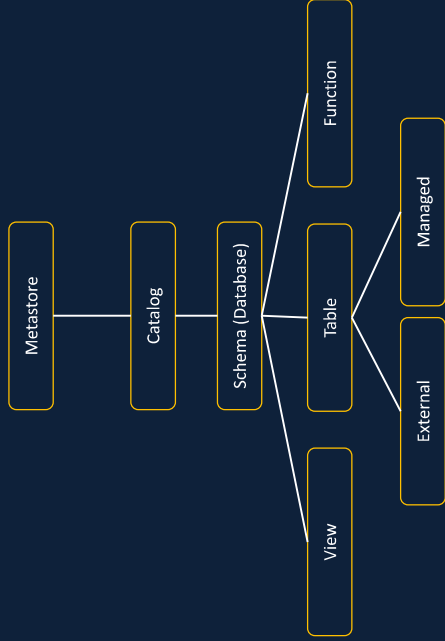


- Managed tables can only be Delta format
- Stored in the default storage
- Deleted data retained for 30 days
- Benefits from automatic maintenance and performance improvements

Unity Catalog Object Model



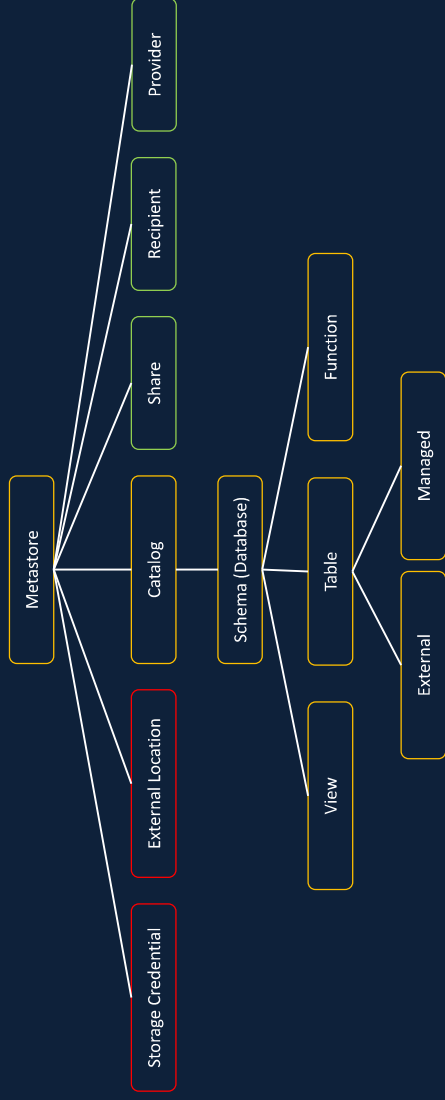
Unity Catalog Object Model



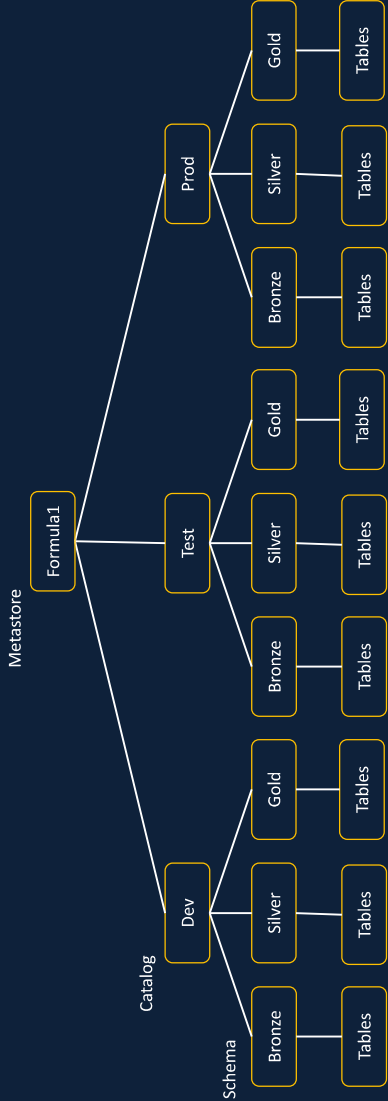
`SELECT * FROM catalog.schema.table;`

`SELECT * FROM catalog.schema.table;`

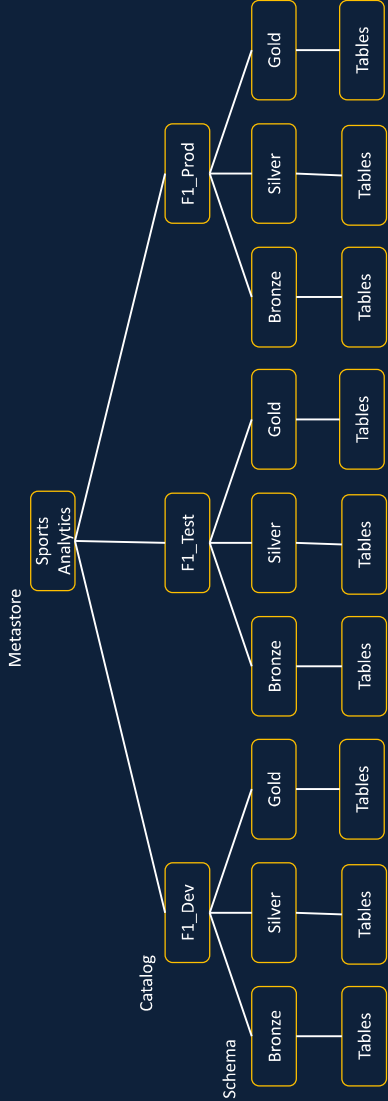
Unity Catalog Object Model



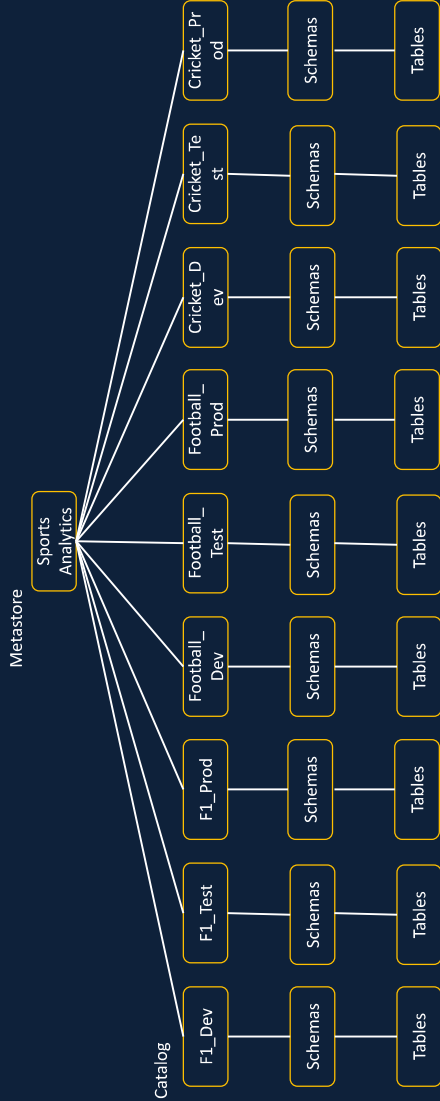
Unity Catalog Object Model



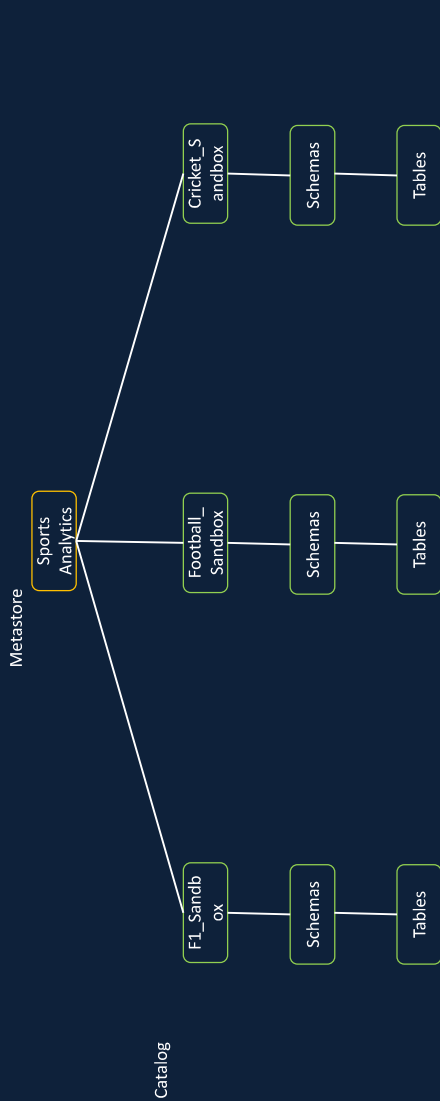
Unity Catalog Object Model



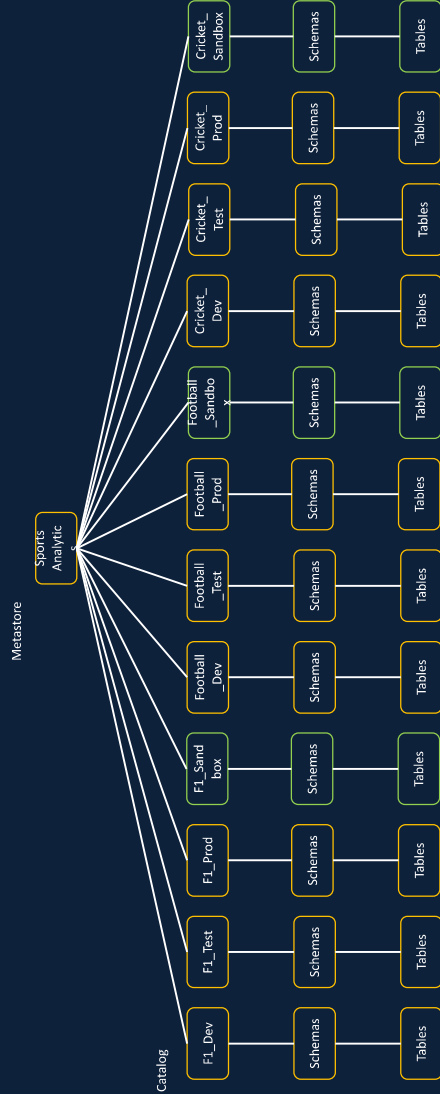
Unity Catalog Object Model



Unity Catalog Object Model



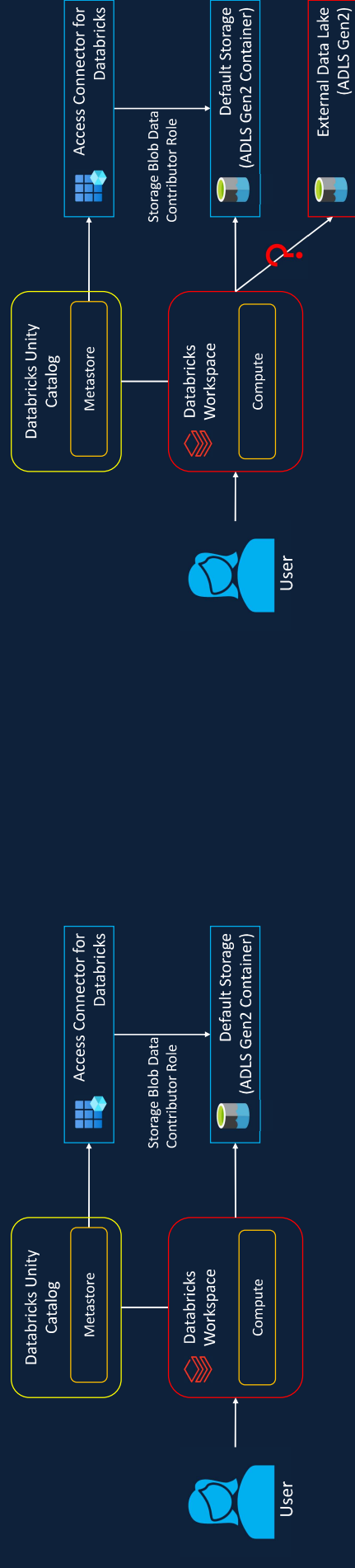
Unity Catalog Object Model



Accessing External Locations

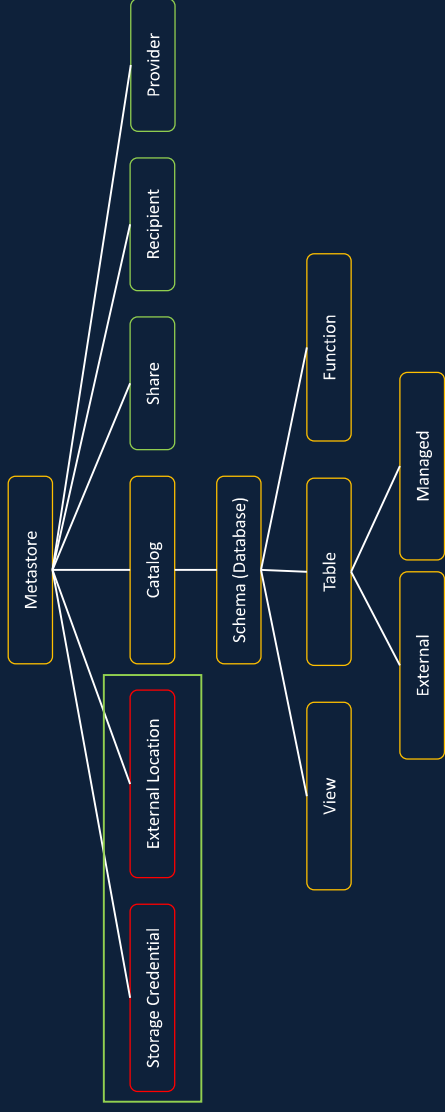


Unity Catalog Configuration



Accessing External Locations

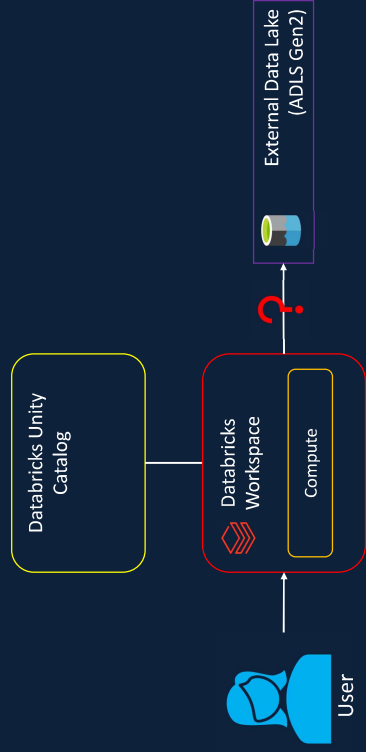
Unity Catalog Object Model



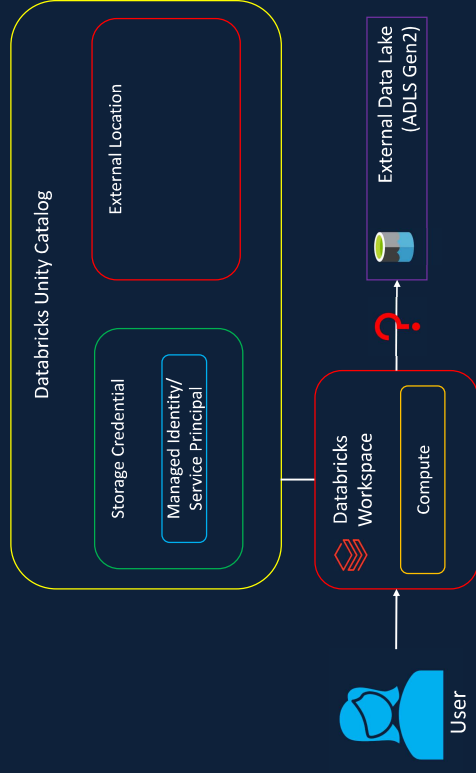
Storage Credential/ External Location



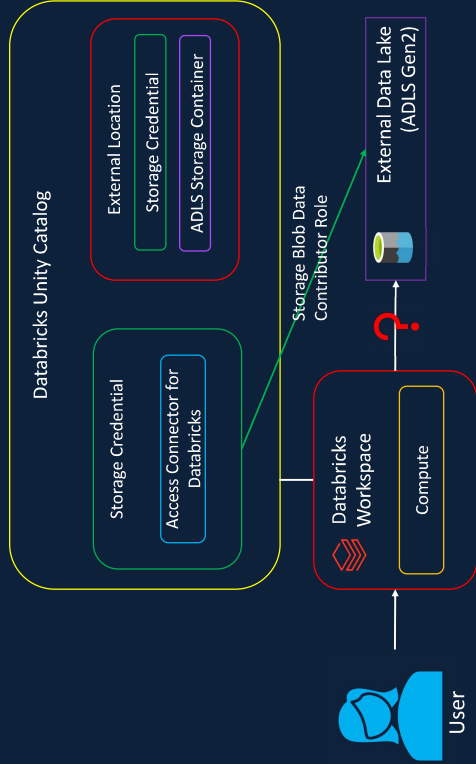
Accessing External Locations



Accessing External Locations

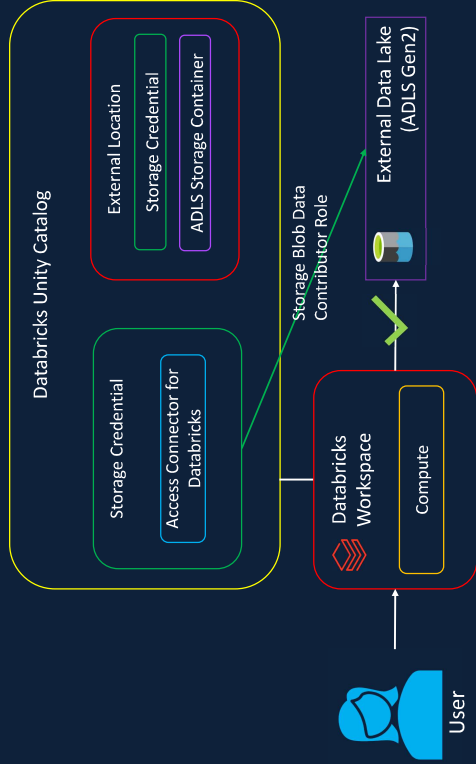


Accessing External Locations

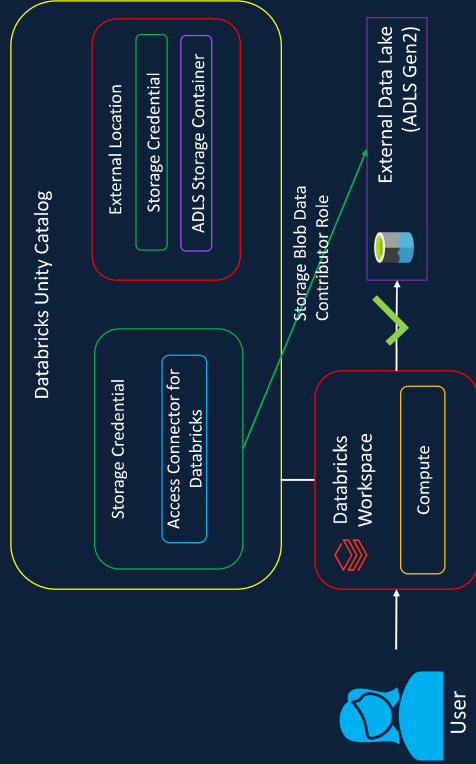


- Create Access Connector
- Create Azure Data Lake Storage
- Assign Storage Blob Data Contributor role
- Create Storage Credential
- Create External Location

Accessing External Locations

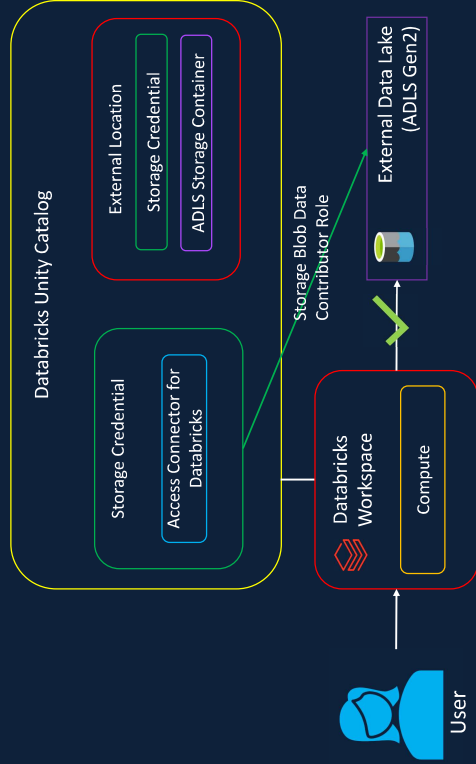


Creating Storage Credential



- Create Access Connector
- Create Azure Data Lake Storage
- Assign Storage Blob Data Contributor role
- Create Storage Credential
- Create External Location

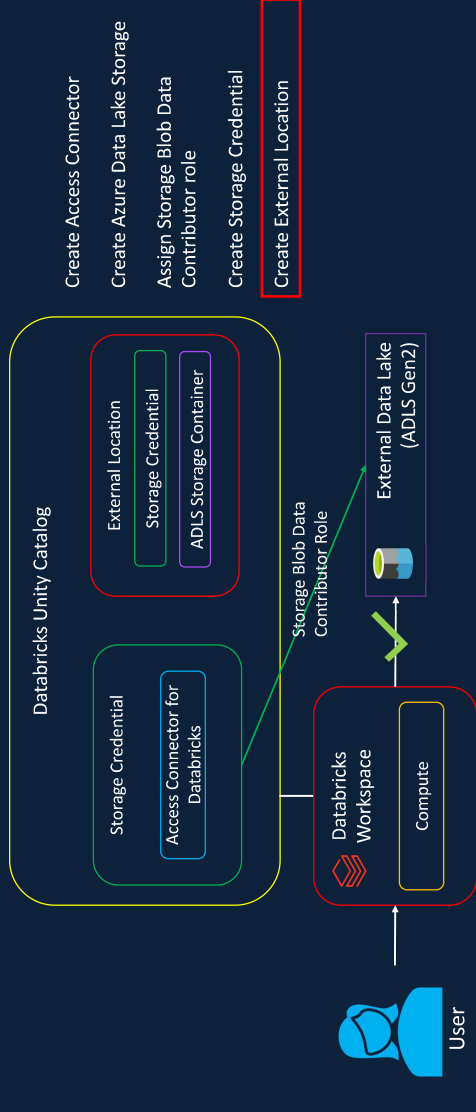
Creating Storage Credential



- Create Access Connector
- Create Azure Data Lake Storage
- Assign Storage Blob Data Contributor role
- Create Storage Credential
- Create External Location

Creating External Location

Mini Project - Overview



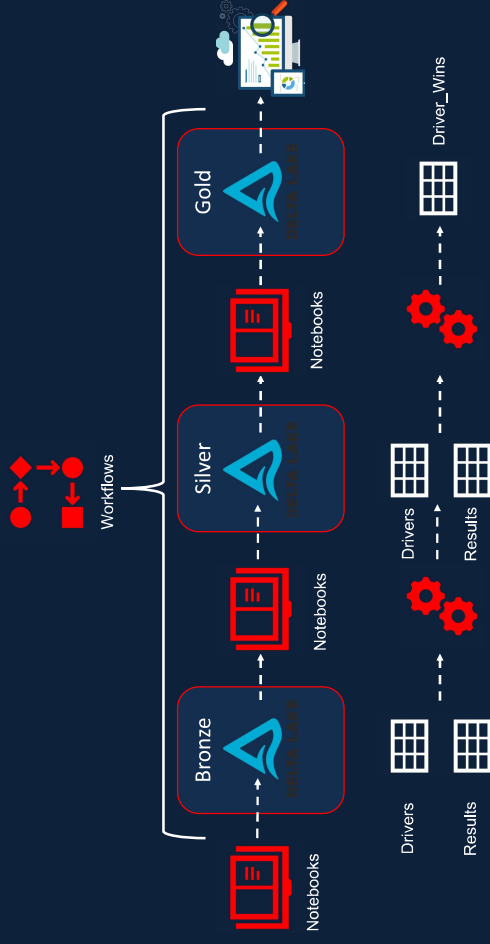
Create Access Connector

Create Azure Data Lake Storage

Assign Storage Blob Data Contributor role

Create Storage Credential

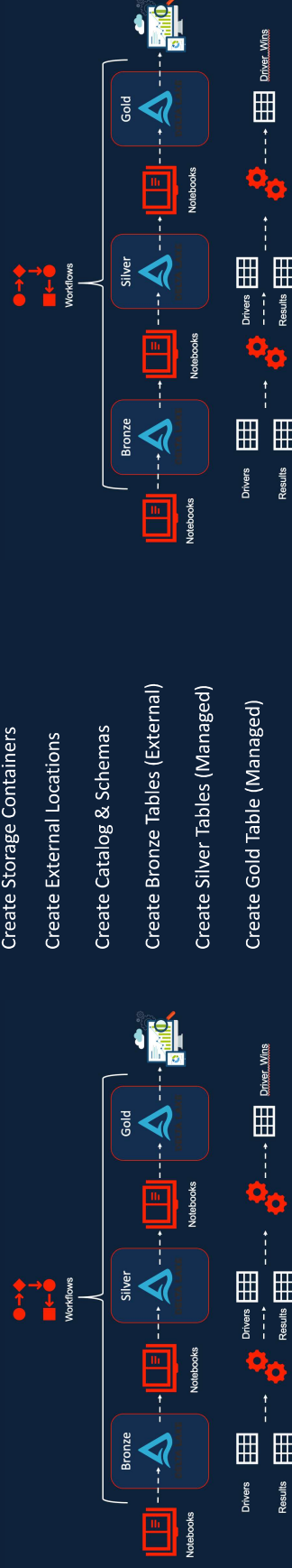
Create External Location



Mini Project - Overview

Mini Project – Create External Locations

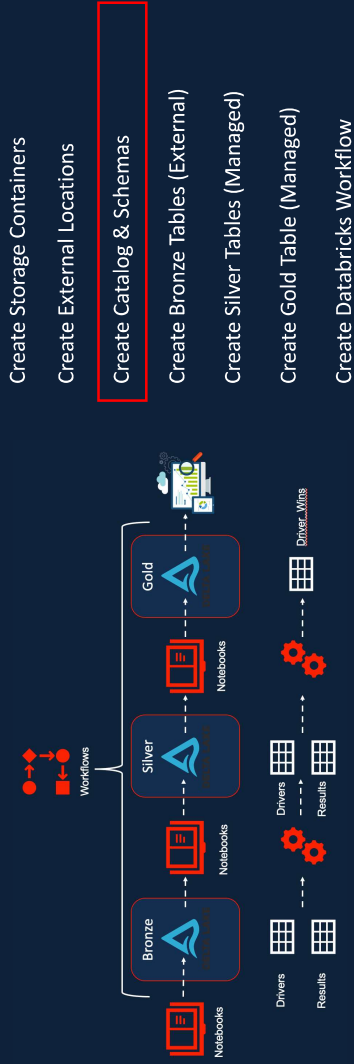
- Create Storage Containers
- Create External Locations
- Create Catalog & Schemas
- Create Bronze Tables (External)
- Create Silver Tables (Managed)
- Create Gold Table (Managed)
- Create Databricks Workflow



- Create Storage Containers
- Create External Locations

- Create Catalog & Schemas
- Create Bronze Tables (External)
- Create Silver Tables (Managed)
- Create Gold Table (Managed)
- Create Databricks Workflow

Mini Project – Create Catalogs & Schemas



Create Storage Containers

Create External Locations

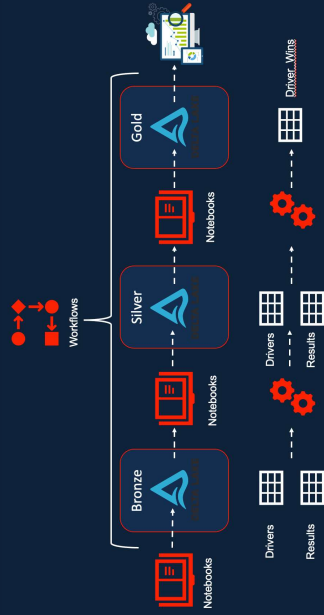
Create Catalog & Schemas

Create Bronze Tables (External)

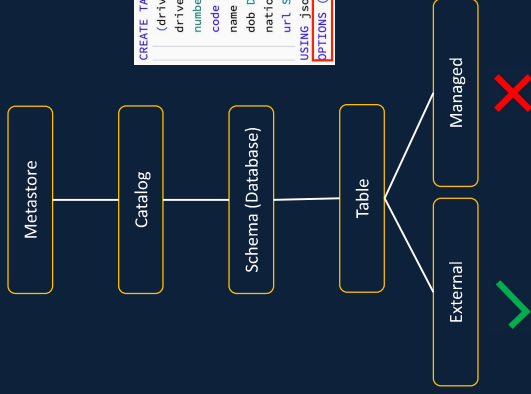
Create Silver Tables (Managed)

Create Gold Table (Managed)

Create Databricks Workflow



Mini Project – Create Catalogs & Schemas



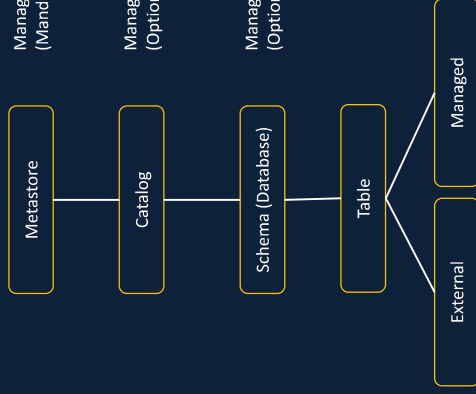
```
CREATE TABLE IF NOT EXISTS formula1_dev.bronze.drivers
(driverId INT,
driverRef STRING,
number INT,
code STRING,
name STRUCT<forename: STRING, surname: STRING>,
dob DATE,
nationality STRING,
url STRING)
USING json
OPTIONS (path "abfss://bronze@databricksoursevtl1.dfs.core.windows.net/drivers.json")
```

Mini Project – Create Catalogs & Schemas



```
CREATE TABLE formula1_dev.silver.drivers
AS
SELECT driverId AS driver_id,
driverRef AS driver_ref,
number,
code,
concat(name.forename, ' ', name.surname) AS name,
dob,
nationality,
current_timestamp() AS ingestion_date
FROM formula1_dev.bronze.drivers;
```

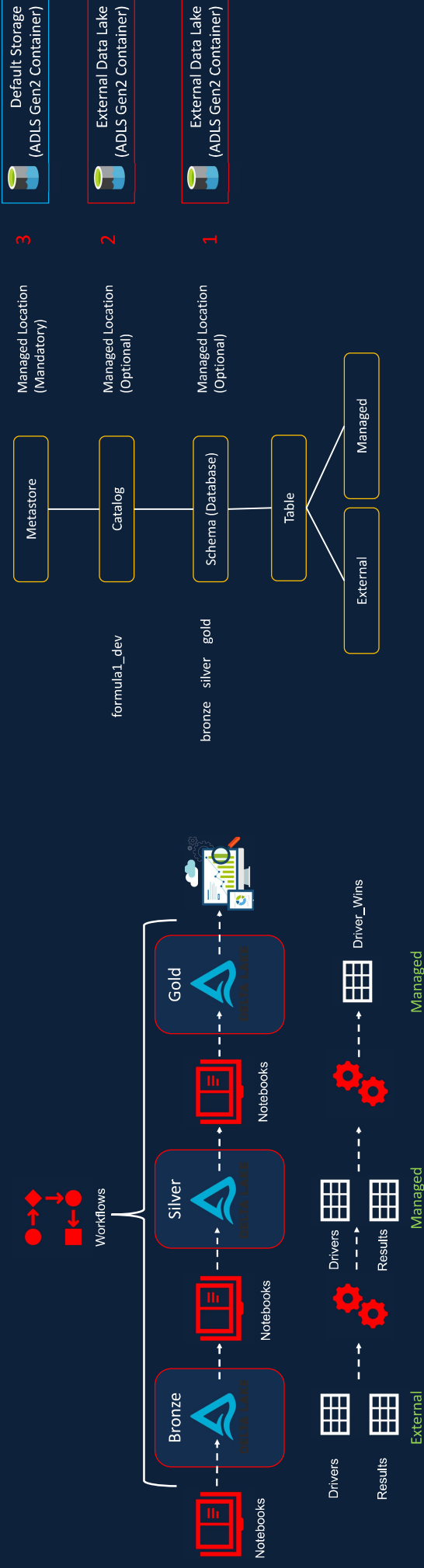
Mini Project – Create Catalogs & Schemas



Managed Location (Mandatory)	Managed Location (Optional)	Managed Location (Optional)
Default Storage (ADLS Gen2 Container)	External Data Lake (ADLS Gen2 Container)	External Data Lake (ADLS Gen2 Container)

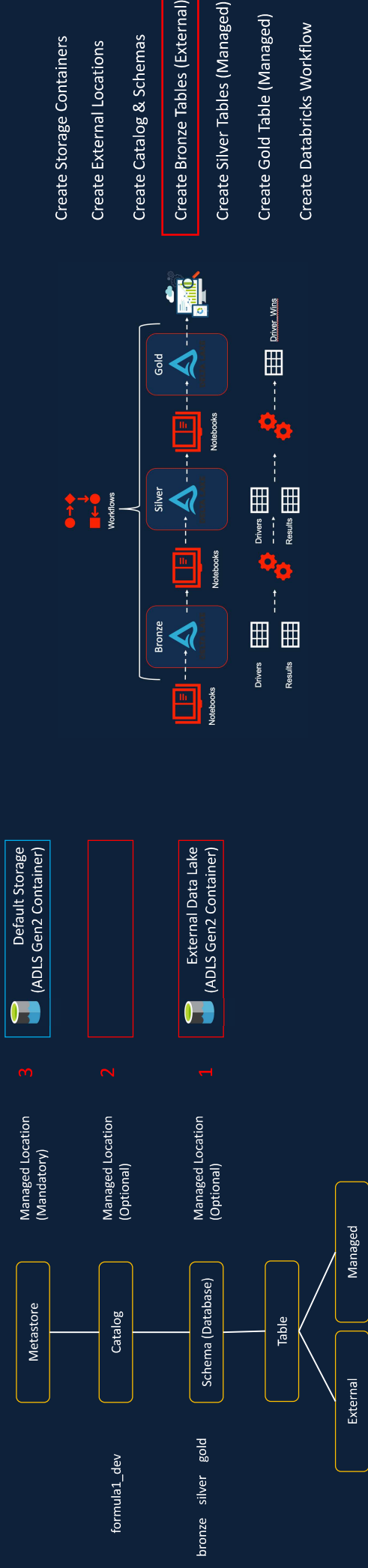
Mini Project – Create Catalogs & Schemas

Mini Project – Create Catalogs & Schemas

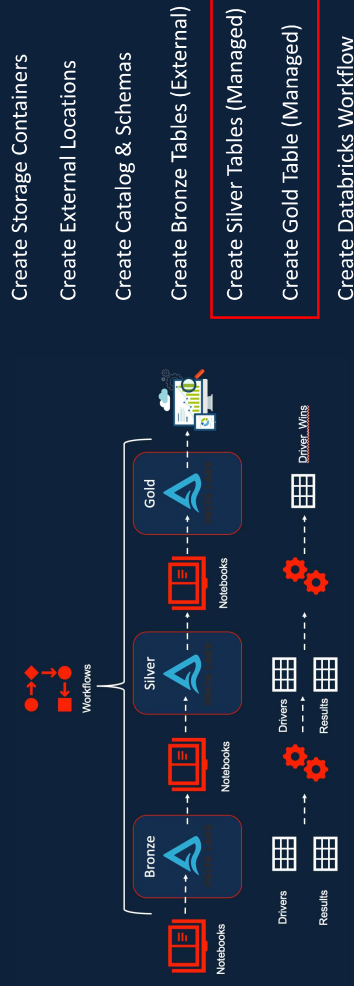


Mini Project – Create Catalogs & Schemas

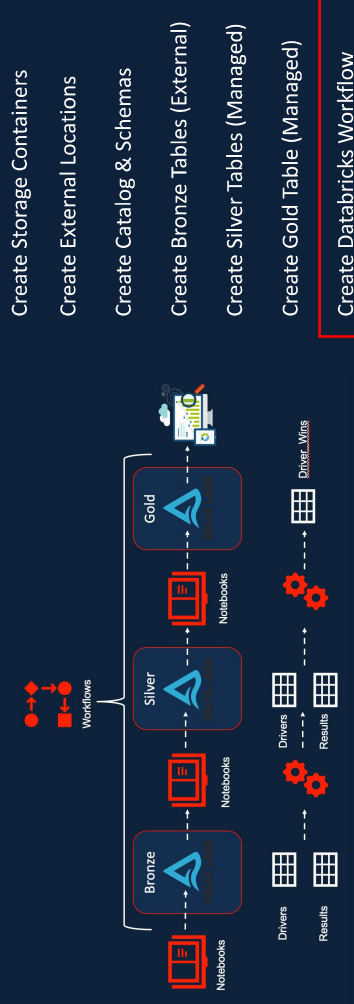
Mini Project – Create External Tables



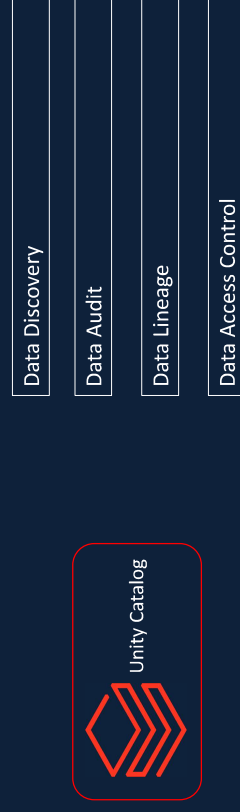
Mini Project – Create Managed Tables



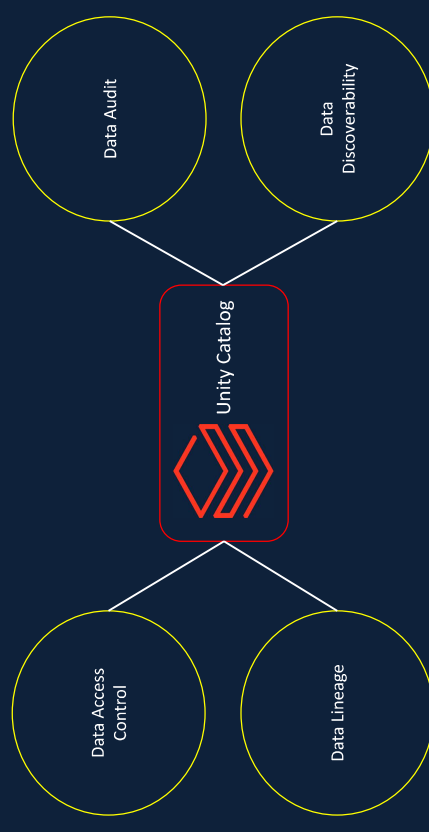
Mini Project – Create Workflow



Unity Catalog – Key capabilities

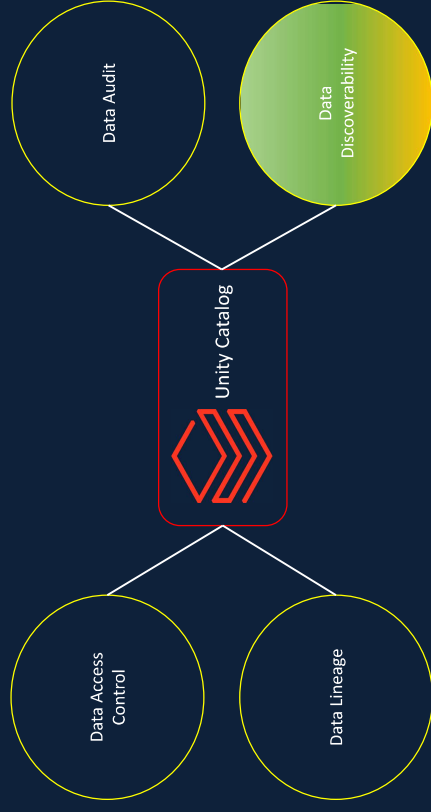


Unity Catalog – Data Discoverability



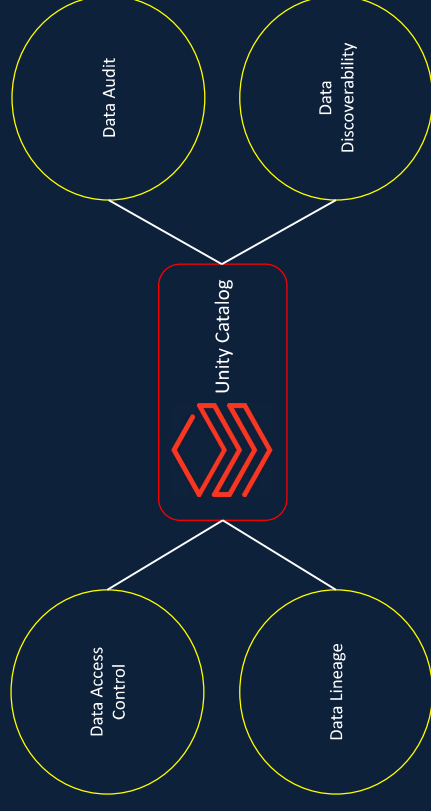
Unity Catalog is a Databricks offered unified solution for implementing data governance in the Data Lakehouse

Unity Catalog – Data Discoverability



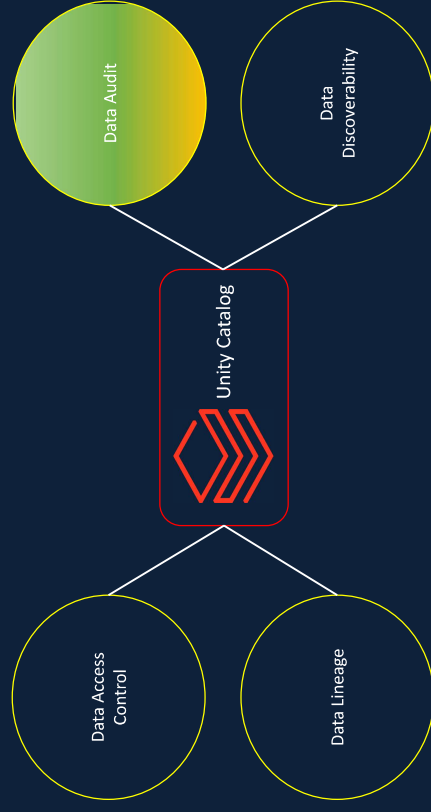
Unity Catalog is a Databricks offered unified solution for implementing data governance in the Data Lakehouse

Unity Catalog – Data Audit



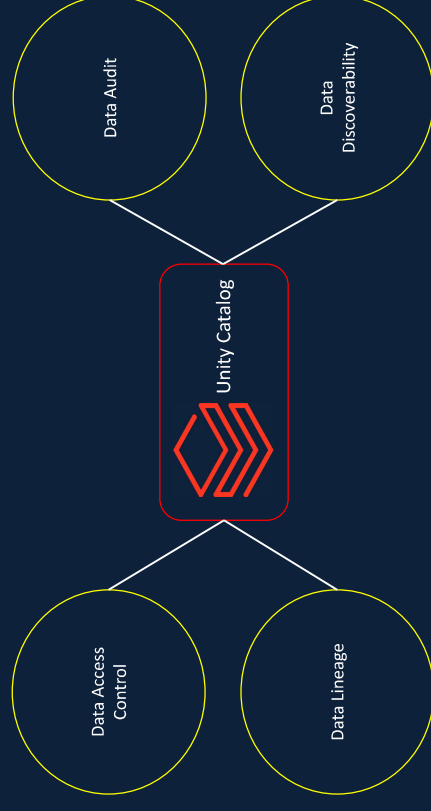
Unity Catalog is a Databricks offered unified solution for implementing data governance in the Data Lakehouse

Unity Catalog – Data Discoverability



Unity Catalog is a Databricks offered unified solution for implementing data governance in the Data Lakehouse

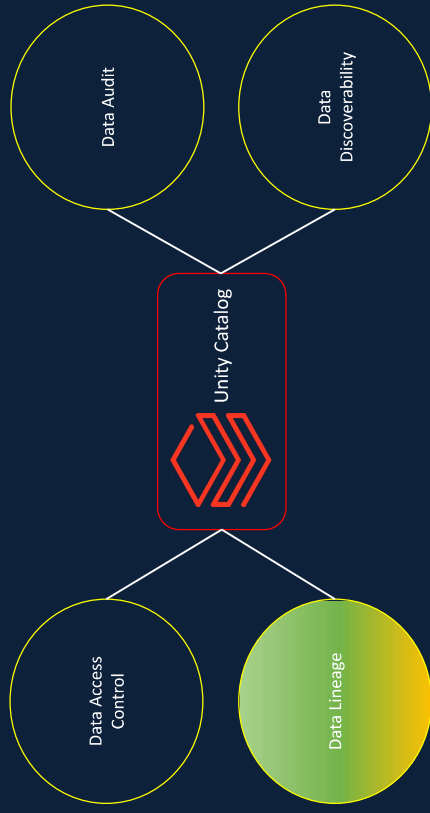
Unity Catalog – Data Lineage



Unity Catalog is a Databricks offered unified solution for implementing data governance in the Data Lakehouse

Unity Catalog – Data Lineage

Data Lineage



Unity Catalog is a Databricks offered unified solution for implementing data governance in the Data Lakehouse

Data Lineage is the process of following/ tracking the journey of data within a pipeline.

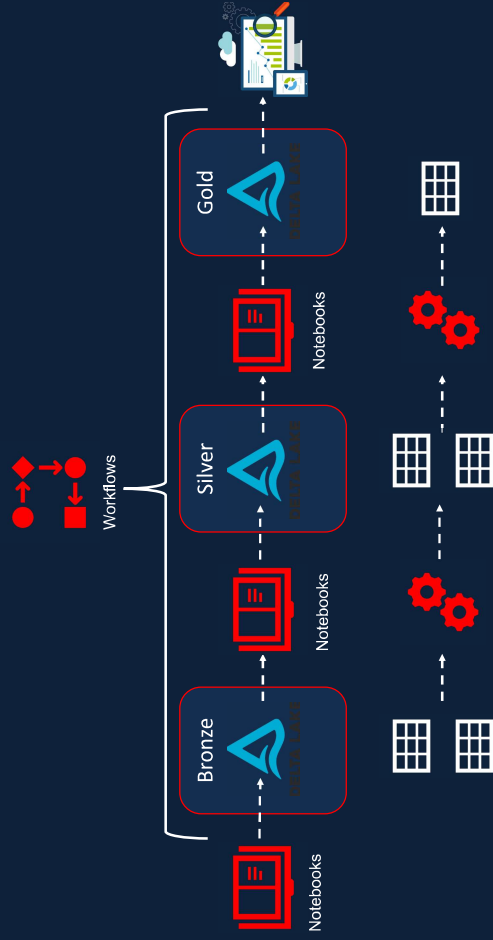
What is the origin

How has it changed/ transformed

What is the destination

Data Lineage

Data Lineage - Benefits



Root cause analysis

Improved impact analysis

Trust & Reliability

Better compliance

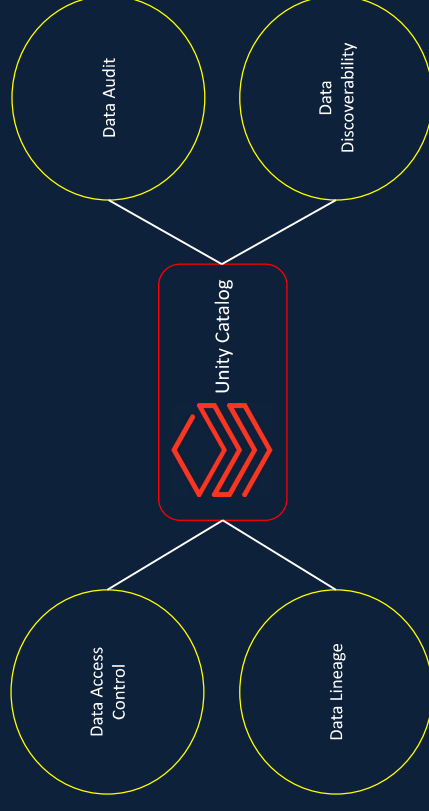
Data Lineage - Limitations

Only available for tables registered in Unity Catalog Metastore

Available only for the last 30 days

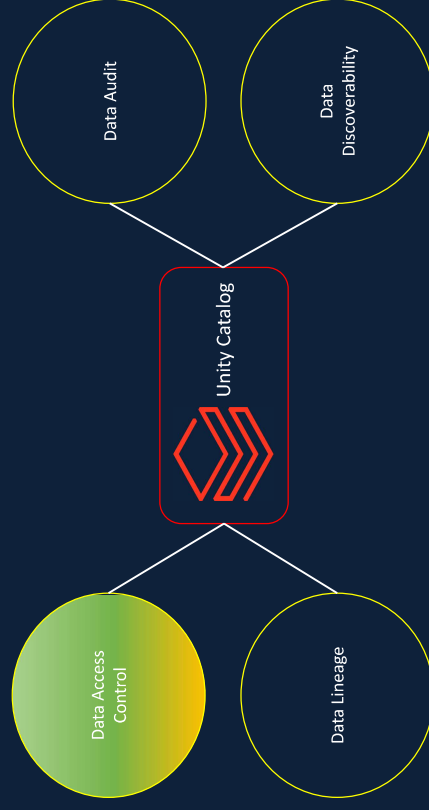
Limited column level lineage

Unity Catalog – Data Access Control



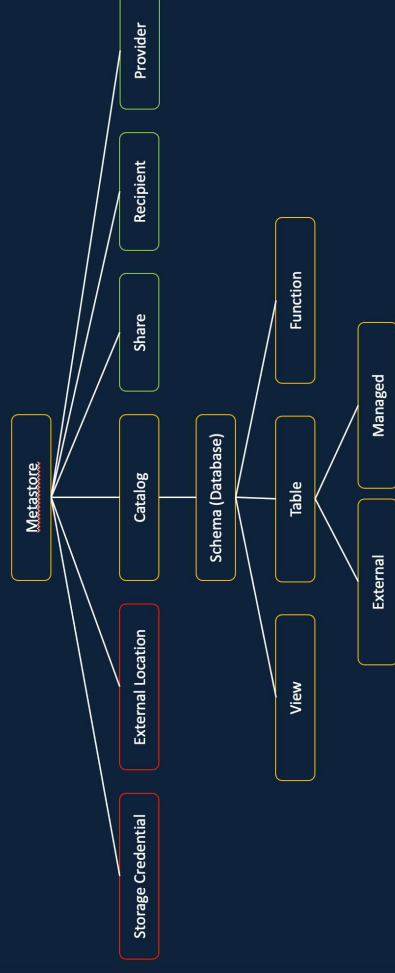
Unity Catalog is a Databricks offered unified solution for implementing data governance in the Data Lakehouse

Unity Catalog – Data Access Control



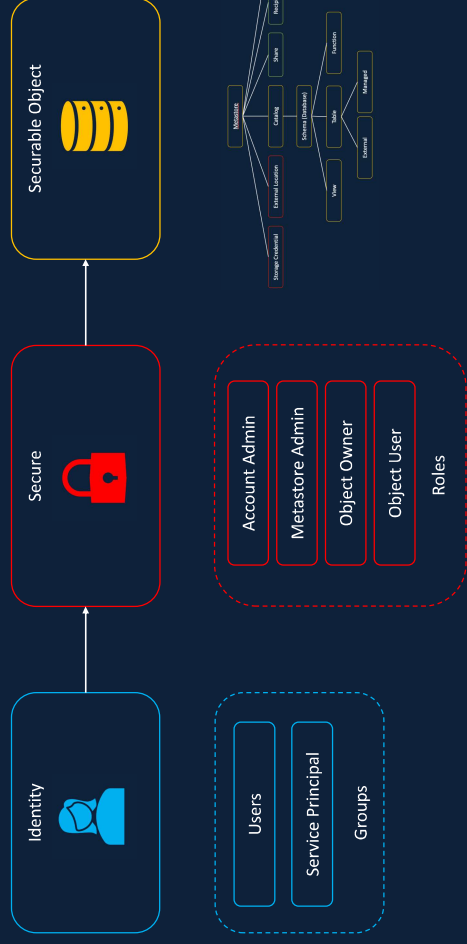
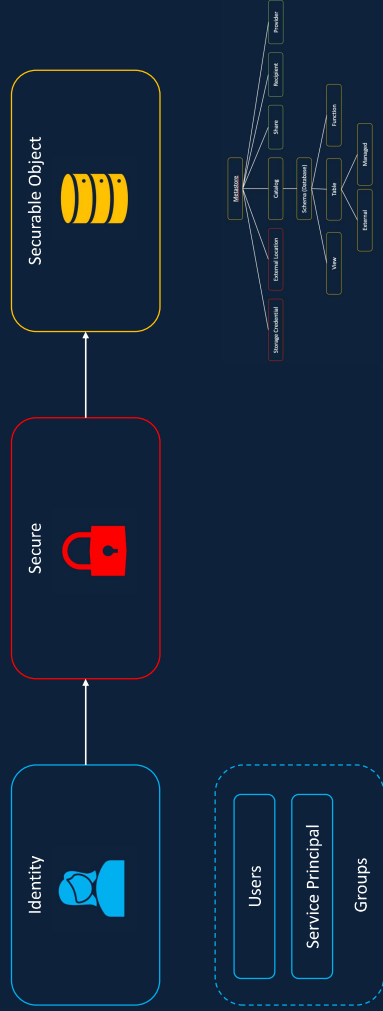
Unity Catalog is a Databricks offered unified solution for implementing data governance in the Data Lakehouse

Unity Catalog - Securable Objects

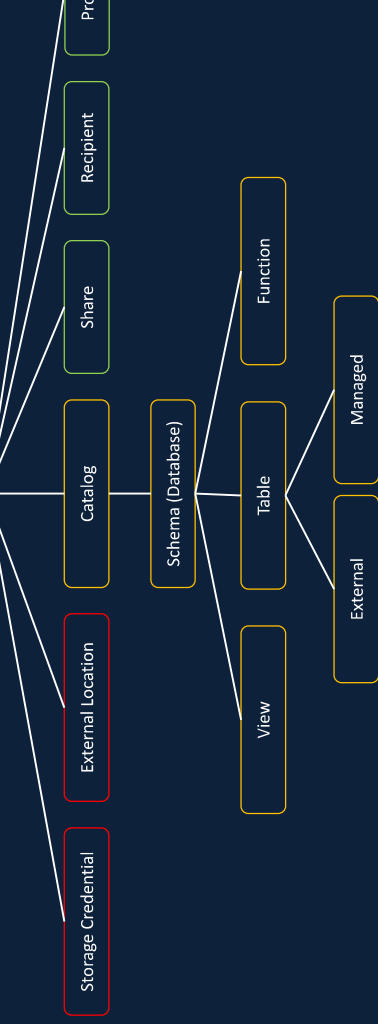
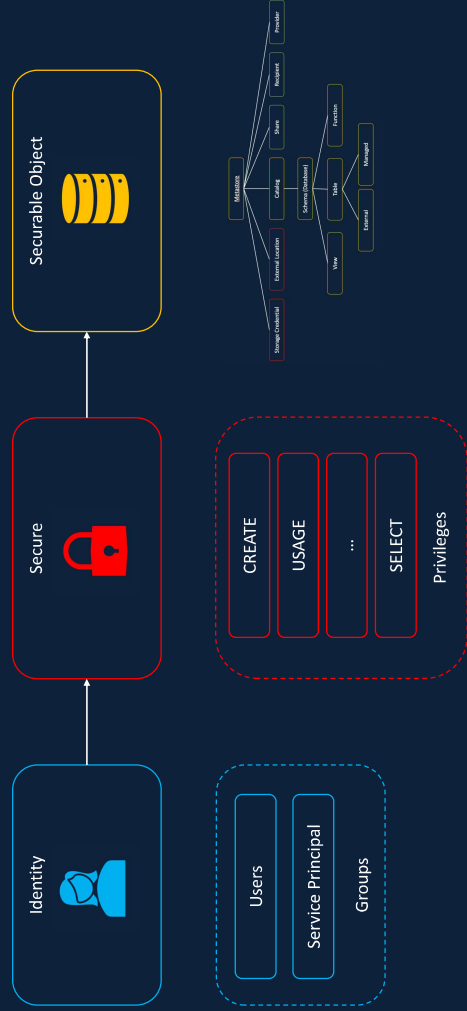


Unity Catalog - Security Model

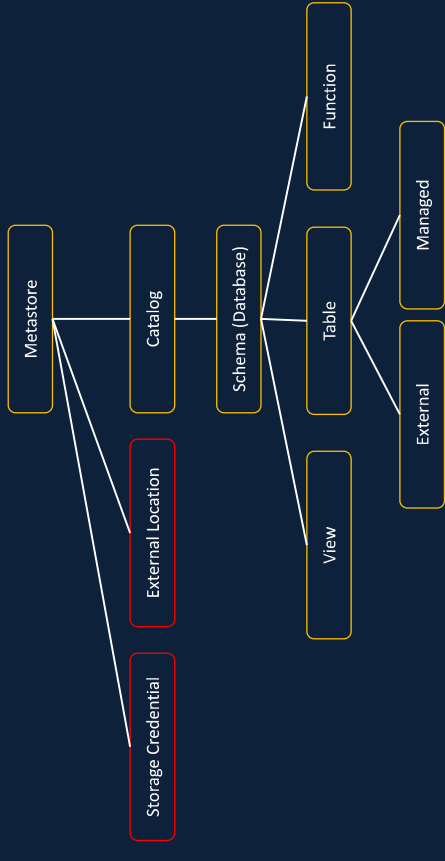
Security Model – Role Based Access



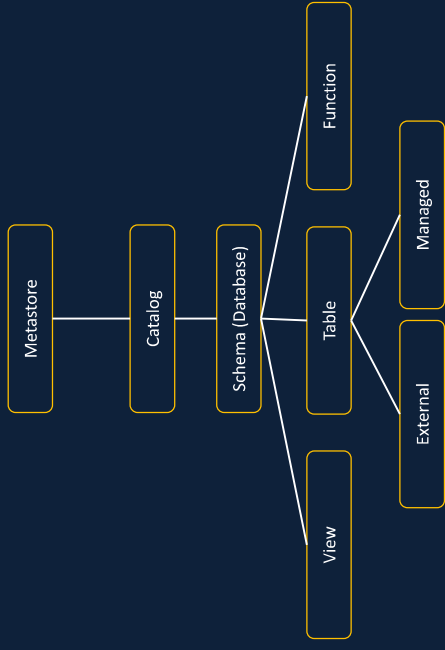
Security Model – Access Control List



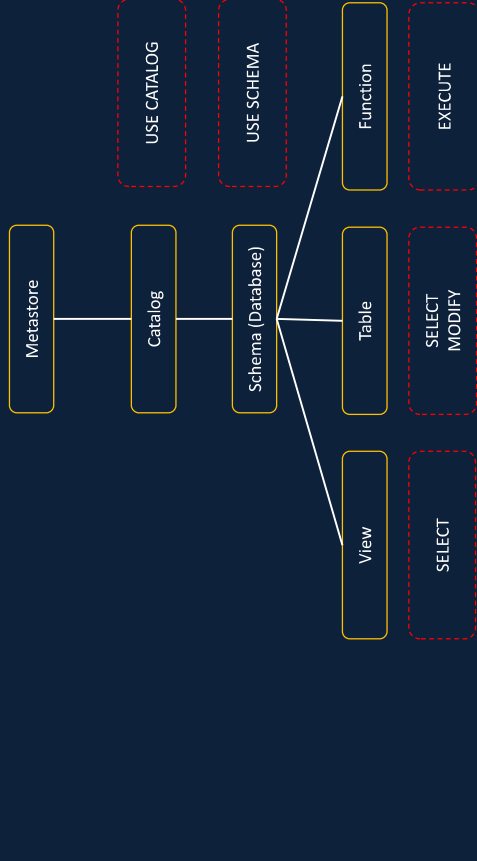
Security Model – Access Control List



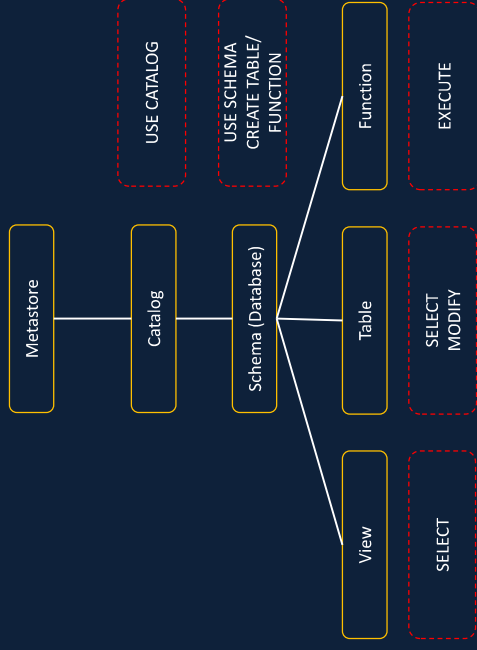
Security Model – Access Control List



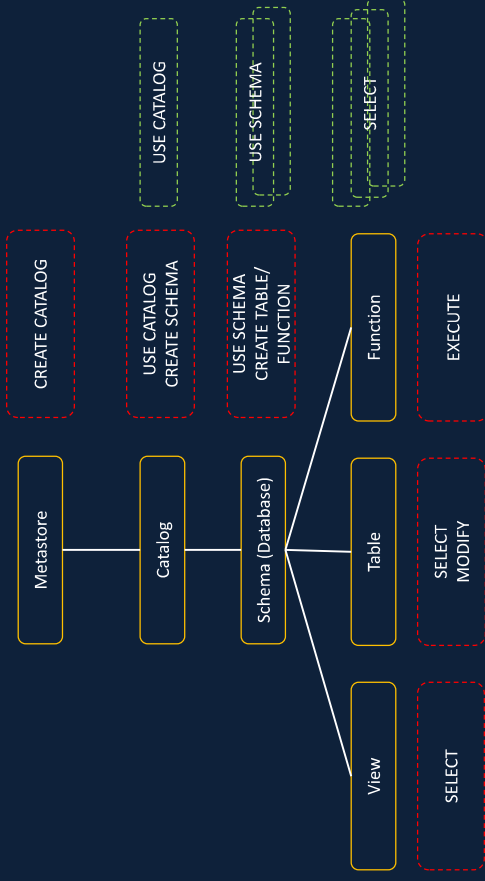
Security Model – Access Control List



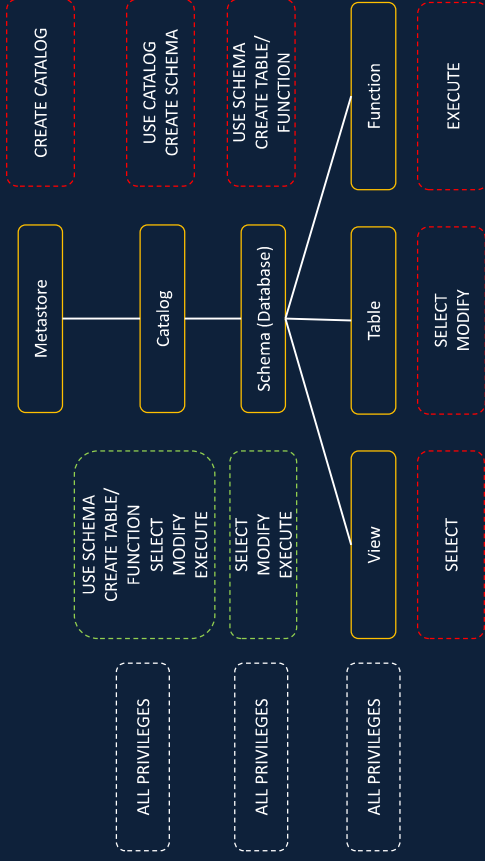
Security Model – Access Control List



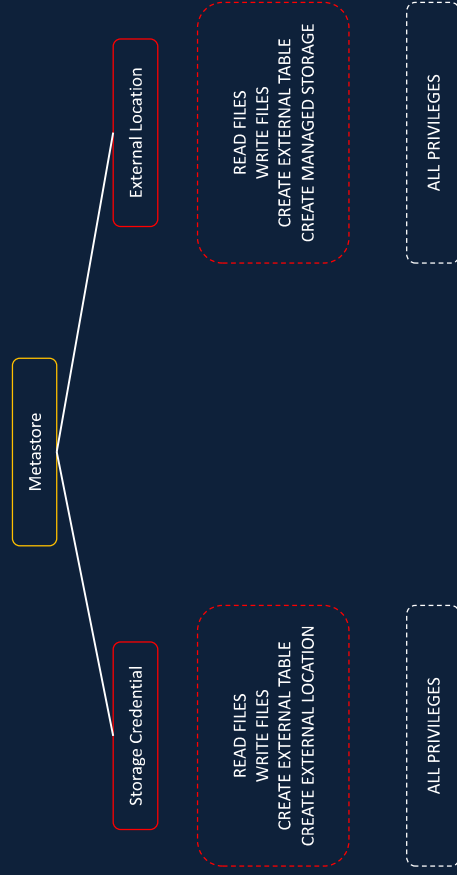
Security Model – Access Control List



Access Control List – Inheritance Model



Security Model – Access Control List



Congratulations!
&
Thank you

Feedback

Ratings & Review

Thank you
&
Good Luck!

Document History

Version	Date	Description
1	July 2021	Initial Version
2	Dec 2022	Sections 3 to 5 updated for UI Changes
3	Mar 2023	Revamped Databricks Mounts and Access to ADLS sections as a whole
4	May 2023	Addition of Unity Catalog